



colegio
CALASANZ
SALAMANCA

Paseo de Canalejas, 139-159
37001 Salamanca
Tel.: 923 26 79 61 | Fax: 923 27 04 54
www.calasanzsalamanca.es
CENTRO PRIVADO CONCERTADO

Skhatoll

CFGS DAW

CURSO 2025/2026

Francisco Gerardo Ripoll Felipe

David Gutiérrez Salvat

Felix Samuel Rojas Ñacari

COLEGIO CALASANZ SALAMANCA

Índice

1. Introducción.	3
2. Descripción de la aplicación.	4
3. Plan de empresa.	7
4. Tecnologías escogidas y justificación.	17
5. Diseño de la aplicación.	19
5.1. Diagramas y definición de casos de uso	19
5.2. Diagramas de clase	39
5.3. Modelo de base de datos y Modelo entidad relación	55
6. Arquitectura de la aplicación	57
6.1. Estructura del proyecto	58
6.2. Librerías externas utilizadas	59
7. Manual de despliegue	60

1. Introducción.

Skhatoll es una plataforma digital para poder jugar y consultar información sobre el juego de mesa de *Los Hombres Lobo de Castronegro*, un juego grupal de roles ocultos, es decir, un juego en el que a cada usuario se le dará una carta con el personaje que tiene que interpretar junto con su descripción, objetivos y cualidades. En el caso del juego de Los Hombres Lobo, mientras el jugador representa su papel y sigue las pautas del narrador deberá cumplir ciertas metas para alzarse con la victoria, ya sea individualmente o en equipo.

En la aplicación de Skhatoll, uno de los jugadores creará una sala donde pueda jugar con otros usuarios. El jugador que crea la sala podrá decidir si ser el narrador de la partida o un jugador normal como el resto, este detalle es importante porque el narrador es el único que conoce qué papel interpreta cada jugador además de ser el que va contando la historia, marca los tiempos, sucesos, interacciones con los personajes, etc.

Sobre el juego, Los Hombres Lobo de Castronegro: en este juego se narra la historia del pueblo de Castronegro, una pequeña villa donde algunos aldeanos fueron infectados por la mordedura de un hombre lobo. Cuando cae la noche, los aldeanos infectados se convierten en lobos y devoran a los habitantes del pueblo por lo que, durante el día, los jugadores deberán averiguar cuáles de sus vecinos son hombres lobos para votar en contra de ellos y eliminarlos. Ganará el bando que elimine a todos los miembros del equipo contrario.

Sobre el nombre, Skhatoll: es una mezcla de los nombres nórdicos de *Sköll* y *Hati*, los lobos gemelos hijos de *Fenrir*, los cuales persiguen sin parar al Sol y a la Luna para comérselos lo que da lugar a los ciclos de día y noche. El comienzo del *Ragnarök*, el fin del mundo nórdico, arrancará después de que estos lobos alcancen a sus presas y las devoren. Esta historia nos sirvió como metáfora de lo que ocurre durante las partidas del juego en las que los aldeanos luchan día y noche por sobrevivir contra los lobos.

Origen de la idea: nace a raíz de las partidas físicas en las que muchas veces a la hora de jugar los roles de los personajes se revelan “sin querer” ya que las cartas pueden verse por despiste del jugador o no saben qué hace su personaje porque no viene una descripción en su carta y tienen que volver a preguntar al narrador, o durante la partida hacen algún ruido o gesto que puede revelar su identidad a otros jugadores, etc. Es decir, **lo que queremos desarrollar** es una plataforma que sirva para jugar de manera más “segura” a Los Hombres Lobo de Castronegro, sin dar pistas accidentales, que oculte mejor el papel de cada jugador y que sirva como guía del juego, además de ayudar y dar soporte al narrador durante las partidas convirtiendo su trabajo en algo más ameno y divertido.

2. Descripción de la aplicación.

La aplicación consiste en una plataforma digital para la gestión de partidas del juego **Los Hombres Lobo de Castronegro**, permitiendo jugar de forma presencial.

El sistema está orientado a 2 tipos de usuario: narrador y jugador, ambos identificados mediante una cuenta registrada.

Principios generales de una partida de Los Hombres Lobo de Castronegro

Bajo la dirección de un narrador, una partida se desarrolla alternando una serie de días y noches. El objetivo de cada jugador dependerá de su identidad secreta y bando.

Durante las noches, todos los jugadores duermen (cierran los ojos en partidas presenciales) y después, por turnos, el narrador indicará a ciertos personajes que abran los ojos, que despierten, para utilizar sus poderes y volverse a dormir. Una vez se hayan llamado a todos los personajes nocturnos, el narrador indicará que se ha hecho de día para que todos abran los ojos (la aldea se despierta).

Durante el día, al despertar la aldea se revelan las víctimas de los personajes nocturnos, en caso de que las haya. A continuación, los jugadores supervivientes debaten y eligen por votación a uno entre ellos con la esperanza de encontrar al culpable o a los culpables del asesinato. El jugador elegido revelará su identidad y quedará eliminado de la partida.

Funcionalidades de la aplicación

Gestión de usuarios

La aplicación permite:

- Registro de nuevos usuarios.
- Inicio y cierre de sesión.

Gestión de salas

Desde la opción de Sala de Juegos del menú, un usuario tiene la capacidad de:

- Crear sala.
 - Asignar el rol de narrador a otro jugador o a sí mismo.
 - Visualizar y compartir el código de la sala.
 - Gestionar la entrada y salida de jugadores.
 - Cerrar la sala manualmente.
- Unirse a una sala mediante un código.
 - Visualizar la lista de jugadores conectados.
 - Abandonar la sala antes del inicio de la partida.

Desarrollo de la partida

Una vez iniciada la partida:

- Los roles se asignan de forma automática y aleatoria.
- El narrador controla el paso entre día y noche.
- El narrador puede habilitar las elecciones para elegir a un alcalde.
- El narrador puede habilitar las votaciones para linchar a un personaje.
- El sistema habilita las habilidades según el rol y la fase.
- Se muestran mensajes automáticos de estado.

Personajes jugables

Para el lanzamiento de esta aplicación hemos decidido escoger a estos 8 personajes, 8 roles fundamentales para poder jugar cualquier partida. Los personajes y sus poderes son los siguientes:

- **Aldeano:** personaje básico sin habilidades especiales.
- **Bruja:** puede resucitar y envenenar a otros personajes utilizando cada poder una vez por partida.
- **Hombre Lobo:** puede devorar a otro personaje una vez por noche.
- **Vidente:** puede ver el rol de otro personaje una vez por noche.
- **Cazador:** cuando muere, puede matar a otro personaje seleccionado por él mismo.
- **Niña:** puede espiar por la noche para descubrir quién puede ser un lobo.
- **Niño Salvaje:** elige a un mentor y si ese mentor muere, el niño se convertirá en lobo.
- **Cupido:** una vez por partida puede hacer que se enamoren 2 jugadores.

Consulta de información

En estas páginas principalmente se puede consultar información relacionada con el juego de mesa además de servir de guía para orientar a los jugadores durante las partidas.

- **Pantalla de Inicio:** formada por 3 apartados con botones que permiten acceder a otras secciones de la web a los que se hace referencia.
 - Sección lateral izquierda que habla sobre el juego de Los Hombres Lobo de Castronegro.
 - Sección central que habla sobre la aplicación de Skhatoll.
 - Sección lateral derecha que habla sobre curiosidades.

- **Pantalla de Reglas y Juegos:** aquí se recogen las reglas principales del juego junto con una guía de seguimiento de la partida. Incluye los apartados de:
 - Información importante para comprender el juego (votaciones y fases).
 - Tabla de distribución de personajes y consejos de preparación.
 - Organización de los turnos de partida.
 - Carrusel de recomendaciones de otros juegos de mesa.
- **Pantalla de Personajes:** en esta vista se da información sobre todo lo relacionado con los personajes. Se organiza de la siguiente manera:
 - **Sección de personajes:** definición, clasificación y sus funciones.
 - **Sección del narrador:** descripción de la figura del narrador.
 - Listas desplegables con los personajes del juego clasificados por bandos. Cada personaje tiene su descripción y explicación de poderes.

3. Plan de empresa.

Planificación del proyecto:

- **Duración estimada:** 20 semanas (aproximadamente 5 meses)
- **Equipo:** 3 personas
- **Metodología:** Iterativa
- **Fases del proyecto**
 - **Análisis y planificación:**
 - Definición de objetivos.
 - Investigación sobre aplicaciones similares.
 - Reparto de tareas y organización del trabajo.
 - Elaboración de diagramas y diseño inicial.
 - **Diseño de la aplicación:**
 - Diseño de interfaz y experiencia de usuario.
 - Modelado de base de datos.
 - Diseño de arquitectura MVC desacoplada.
 - Definición de endpoints y flujo de datos.
 - **Desarrollo del backend:**
 - Implementación de la API REST.
 - Configuración de Spring Boot y Hibernate.
 - Gestión de usuarios y autenticación.
 - Implementación de lógica de juego y WebSockets.
 - **Desarrollo de frontend:**
 - Creación de vistas y componentes con Vue.js.
 - Integración con la API.
 - Gestión del estado del juego.
 - Diseño responsive y optimización visual.
 - **Pruebas y control de calidad**
 - Pruebas funcionales.
 - Corrección de errores.
 - Validación de partidas multijugador.
 - Revisión de experiencia de usuario.
 - **Despliegue y documentación:**
 - Configuración Docker.
 - Preparación de la memoria técnica.
 - Elaboración del manual de despliegue.
 - Presentación final del proyecto.

- **Contenido del proyecto**

El proyecto incluye:

- Aplicación web SPA desarrollada con Vue.js.
- Backend desacoplado desarrollado con Spring Boot.
- Sistema de autenticación y gestión de usuarios.
- Gestión de salas y partidas.
- Comunicación en tiempo real mediante WebSockets.
- Base de datos relacional MySQL.
- Sistema de asignación automática de roles.
- Panel de narrador para control de partidas.
- Documentación técnica y diagramas.
- Configuración Docker para despliegue.

- **Secuenciación del desarrollo por orden de prioridad:**

- Configuración inicial del proyecto y repositorio.
- Diseño de base de datos y arquitectura.
- Implementación del sistema de usuarios.
- Desarrollo de gestión de salas.
- Implementación de la lógica principal de partidas.
- Integración de WebSockets y tiempo real.
- Desarrollo de interfaz gráfica.
- Optimización visual y responsive.
- Pruebas, depuración y control de calidad.
- Documentación y despliegue.

- **Procedimientos de actuación o ejecución de las actividades:**

- Organización del trabajo mediante reparto de tareas.
- Uso de Git y GitHub para control de versiones.
- Desarrollo por ramas feature/* e integración continua mediante Pull Requests.
- Revisión conjunta del código antes de fusionar cambios.
- Realización de reuniones periódicas para seguimiento.
- Uso de commits estructurados para mantener trazabilidad.

- **Control de calidad**

Se aplicarán un conjunto de procesos que garantizan que el producto funciona correctamente, es fiable y cumple con los requisitos definidos. No es solo que no existan errores del sistema, sino que la experiencia del usuario sea buena y el sistema sea robusto. Algunos de estos procesos son:

- **Pruebas funcionales:** verificar que cada funcionalidad hace lo que debe. Son las pruebas más importantes para un juego multijugador.
- **Pruebas de comunicación en tiempo real:** Verificar que los mensajes lleguen a todos los clientes, que no hay pérdidas de conexión y hay posibilidad de reconexión.
- **Pruebas de concurrencia:** para ver cómo maneja la aplicación el que varios usuarios acudan a los recursos a la vez.
- **Pruebas de interfaz:** que los componentes se renderizan correctamente, que los toasts aparecen, que los roles se muestran bien, que la vista es responsive en móvil.
- **Pruebas de seguridad básica:** para que un jugador solo pueda acceder a lo que le tiene permitido su rol y tipo de jugador y que el JWT se valide correctamente.
- **Métricas de calidad:** pruebas de latencia del WebSockets, tiempo de respuesta de la API, comportamiento con muchos jugadores simultáneos.

Servicio que ofrece nuestra aplicación: como se ha comentado anteriormente, Skhatoll está orientado a ser una página web donde los jugadores puedan jugar de una manera más cómoda, segura e intuitiva al juego de mesa de *Los Hombres Lobo de Castronegro* además de poder utilizarla como guía de consulta de las reglas del juego. Es una aplicación orientada a los servicios de ocio y entretenimiento.

- **Necesidades que puede solventar Skhatoll:**

- Evitar que los roles se revelen accidentalmente durante partidas físicas.
- Facilitar el trabajo del narrador automatizando procesos.
- Ayudar a nuevos jugadores mediante explicaciones y guías.
- Mejorar la organización de las partidas.
- Reducir errores humanos durante las votaciones o acciones nocturnas.
- Permitir partidas más dinámicas e inmersivas.
- Centralizar reglas, personajes e información del juego en una sola plataforma

- **Público objetivo:** principalmente está formado por personas interesadas en los juegos de mesa sociales y de deducción. Otros perfiles de usuarios a los que podría llamarles la atención serían:
 - **Jugadores habituales del juego:** que desean agilizar las partidas y automatizar tareas como la gestión de roles, turnos y normas.
 - **Nuevos jugadores:** que necesitan apoyo mediante guías, explicaciones y consultas rápidas de las reglas.
 - **Grupos de amigos y estudiantes:** que utilizan este tipo de juegos como forma de ocio social y entretenimiento.
 - **Usuarios online:** (*en el caso de implementar esa función*) interesados en jugar partidas a distancia mediante una plataforma web.
 - **Creadores de contenido o comunidades de juegos de mesa,** que pueden utilizar la aplicación para organizar partidas de forma más dinámica.

Estructura organizativa y funciones de cada departamento:

La estructura y distribución de tareas se divide en los siguientes departamentos:

- **Desarrollo frontend:**
 - Diseño de interfaces.
 - Componentes Vue.js.
 - Experiencia de usuario.
 - Integración visual.
 - Diseño y maquetación de imágenes y logo.
- **Desarrollo backend:**
 - API REST.
 - Lógica de negocio.
 - Gestión de salas y partidas.
 - Persistencia de datos.
- **Base de datos y despliegue:**
 - Modelado de datos.
 - Configuración MySQL.
 - Docker y despliegue.
 - Gestión del repositorio y control de versiones.

Oportunidades de negocio y objetivos:

Aunque en un inicio el proyecto está pensado de manera académica, puede evolucionar hacia un objetivo más amplio abarcando hacia un mundo digital y juegos multijugador.

- **Objetivos principales:**
 - Crear una experiencia más cómoda y segura para jugar.
 - Modernizar un juego de mesa clásico mediante herramientas digitales.
 - Ofrecer una plataforma intuitiva para todos los jugadores
 - Desarrollar una base sólida para futuras ampliaciones.

- **Posibles implementaciones futuras:** estas implementaciones que podría tener la aplicación le darían una mayor funcionalidad haciendo que fuera escogida frente a las aplicaciones de la competencia muchas de las cuales no las incluyen. Son las siguientes:
 - **Plataforma online multijugador:** para que los jugadores no tengan que quedar en persona para poder jugar. Unido a ello podría crearse una opción para que la gente se uniera a partidas con jugadores de todo el mundo aleatoriamente.
 - **Aplicación para teléfonos móviles:** que busque la practicidad y la velocidad para poder echar una partida en cualquier lugar.
 - **Sistemas de perfiles y estadísticas:** Para que los jugadores puedan tener un control de sus estadísticas de victorias y derrotas, comprobar con qué personajes juegan mejor, hacer torneos, rankings, etc.
 - **Personalización de partidas:** los jugadores pueden elegir qué personajes meter en sus partidas entre otras opciones que podrían introducirse.
 - **Chat para jugadores:** para que pueda comunicarse mejor durante las partidas online. Habría chats privados como por ejemplo, uno para los hombres lobo.
 - **Narrador IA:** Integración de inteligencia artificial orientada a la narración de la partida además de servir como ayuda ante preguntas que puedan surgir durante las partidas.
 - **Adaptación expansiones del juego:** Los Hombres Lobo de Castronegro tiene expansiones en el formato físico que podrían añadirse a su versión online para que el juego sea más divertido. Estas expansiones son:
 - **Personajes:** ampliación del número de personajes seleccionables durante las partidas. Hay de todos los bandos.
 - **La Aldea:** da la opción de que los personajes puedan vivir en edificios y tener cargos (tabernero, panadero, farmacéutico...), los que permiten a los jugadores tener nuevos poderes y habilidades.
 - **Luna Nueva:** esta expansión incluye eventos que hacen que los jugadores se enfrenten a nuevas situaciones y tramas durante las partidas.

- **Multiplataforma de juegos de mesa:** tanto la imagen de la plataforma como las funcionalidades podrían adaptarse para que fuera una aplicación que incluyera distintos juegos de mesa de características similares como *Blood on the Clock Tower* o *Samurai Sword*.

- **Ventajas competitivas:**

- Interfaz moderna e intuitiva.
- Automatización de procesos del juego.
- Sistema de ayuda y guías integradas.
- Arquitectura escalable y mantenible.
- Comunicación en tiempo real mediante WebSockets.
- Separación clara entre frontend y backend.
- Fácil adaptación a futuras expansiones.
- Posibilidad de implementar funcionalidades comentadas en el punto anterior.

Aplicaciones similares: de este juego, ya existen algunas versiones online como boardgamearena.com o en aplicación como Werewolf EVO o Wolvesville pero o las interfaces son poco intuitivas, carecen de ciertas implementaciones como guías orientativas o no son fieles al 100% a las reglas del juego. Además de que no tiene la mayoría de las ideas de futuras implementaciones que podría tener Skhatoll.

Obligaciones y condiciones de la aplicación: (en caso de lanzar este proyecto como un producto descargable para posibles usuarios).

- **Obligaciones fiscales:**

- Alta como actividad económica. Una Sociedad Limitada (S.L.) sería lo más idóneo con un capital mínimo de 3000€. Puede constituirse con un capital menor pero tendría obligaciones adicionales.
- Alta en el IAE (Impuesto de Actividades Económicas), presentación del IVA trimestralmente (modelo 303) y el IRPF si los creadores son socios autónomos al tener más del 25% de las participaciones en la empresa (modelo 130).
- Si se monetiza la app con publicidad o suscripciones, esos ingresos tributan en el Impuesto de Sociedades (25% general, 15% para empresas de nueva creación los dos primeros años).

- **Permisos legales y autorizaciones:**

- Respetar los derechos de autor relacionados con el juego original pidiendo una licencia de uso (ya sea con Asmodee que es la distribuidora o con los propios creadores del juego).
- Uso legal de imágenes, iconos y recursos gráficos. Pedir permisos en caso de ser imágenes y recursos de otros autores.

- Registro del logo y la marca en la Oficina Española de Patentes y Marcas (OEPM) por unos 150-200€.
- Cumplimiento de normativa de protección de datos revisando el RGPD.
- Inclusión de políticas de privacidad y cookies en caso de despliegue público así como un registro de actividades de tratamiento.
- **Obligaciones laborales:**
 - Organización de tareas de manera equitativa, coordinación entre integrantes y cumplimiento de plazos establecidos.
 - Alta en la Seguridad Social
- **Obligaciones relacionadas con la prevención de riesgos laborales:**
 - Descansos periódicos y formación básica en PRL.
 - Uso adecuado de equipos informáticos.
 - Organización segura del espacio de trabajo.
 - Prevención de fatiga visual.

Viabilidad técnica

- **Recursos humanos:**
 - 3 desarrolladores para Frontend y Backend.
 - Un diseñador gráfico UI/ UX.
 - Reparto equilibrado de tareas.
 - Coordinación mediante GitHub.
 - Estudio de campañas de marketing o contratar a alguien externo.
- **Recursos materiales:**
 - Ordenadores personales.
 - Conexión a internet.
 - Servidor local de pruebas.
 - Docker y herramientas de desarrollo.
- **Control de calidad del proyecto:**
 - Validación de código.
 - Gestión segura de ramas.

- Revisiones de integración.
- Pruebas funcionales y de rendimiento utilizando las distintas pruebas mencionadas en el apartado de control de la calidad.

Viabilidad económica y financiera

- **Presupuestos:** Al tratarse de un proyecto desarrollado con herramientas gratuitas y software open source (Vue.js, Spring Boot, MySQL), los costes principales se concentran en infraestructura, identidad digital y recursos gráficos. A continuación se detallan las opciones y estimaciones para el primer año **(1.569€ - 1.774€)**:
 - **Hosting y despliegues (opciones):**
 - **DigitalOcean:** permite desplegar servidores virtuales (Droplets) desde aproximadamente 5-6€ mensuales, ofreciendo un entorno flexible y escalable compatible con Docker. Es la opción más recomendable para este proyecto por su relación calidad-precio y su documentación.
 - **Render:** Ofrece despliegue sencillo conectado con GitHub y planes gratuitos para pruebas, aunque los servicios permanentes suelen comenzar alrededor de 7€ mensuales por servicio.
 - **Railway:** Orientada a despliegues rápidos mediante Docker y GitHub, con planes básicos desde aproximadamente 5€ mensuales.
 - **Vercel:** para el frontend Vue.js, Vercel dispone de un plan gratuito para proyectos personales y un plan profesional desde 18€ mensuales por usuario, aunque para este proyecto el plan gratuito sería suficiente en fases iniciales.
 - **Dominio web:**
 - 8 y 15 € anuales para dominios **(.com)**
 - 5 y 12 € anuales para dominios **(.es)**
 - El certificado SSL es gratuito mediante Let 's Encrypt.
 - **Base de datos y servidor cloud (15 euros/mensuales):**
 - **Aiven:** actualmente el proyecto lo usa como proveedor de MySQL gestionado, que dispone de un tier gratuito suficiente para desarrollo y pruebas.
 - **En producción,** una base de datos administrada en *DigitalOcean* tendría un coste aproximado de 12-15€ mensuales. Como alternativa, *Railway* y *Render* incluyen bases de datos en sus planes de pago.
 - **Recursos gráficos y multimedia:**
 - **Imágenes:** se recurriría a bancos de imágenes, prompts generados por IA y/ o ilustraciones personalizadas.
 - **Iconografía:** Font Awesome o Bootstrap. Opcional recurrir a iconografía premium como la de Font Awesome (80€ anuales).

- **Recursos de sonido:** podría recurrirse a bancos de sonido gratuitos o pagar entre 50€ y 150€ en licencias únicas en plataformas como Epidemic Sound o Artlist.
 - **Licencias de software de diseño gráfico:** saldría gratis puesto que ya tendríamos una Licencia de *Photoshop e Illustrator*.
- **Sueldos participantes:** en la fase inicial el proyecto se desarrolla como trabajo propio sin retribución económica directa. A medida que el proyecto genere ingresos mediante suscripciones u otros modelos de monetización, los costes de infraestructura escalarán proporcionalmente a la demanda de usuarios y podrán establecerse unos sueldos.
- **Costes tras el primer año (609€ - 1.024€):**
 - A partir del segundo año los costes se reducen casi a la mitad al no tener que afrontar los gastos de puesta en marcha.
 - El mayor ahorro viene de eliminar la gestoría de constitución y el registro de marca, que son pagos únicos.
 - El coste recurrente más relevante a vigilar es la base de datos, que escalaría si el número de usuarios crece significativamente y se necesitará más capacidad de almacenamiento o rendimiento.
- **Necesidades de financiación:** En caso de llevar a cabo el proyecto a un nivel profesional del mercado habría que pedir financiaciones externas. Las opciones más viables para un equipo de estudiantes son el préstamo participativo del ENISA (Empresa Nacional de Innovación) orientado a startups tecnológicas, o la aportación de los propios socios dividida entre tres (500-800€ por persona). Si el proyecto escala y tiene usuarios activos, podrías buscar inversión en aceleradoras como Wayra o Lanzadera.
- **Ayudas y subvenciones:**

Para visiones futuras se podrían solicitar:

- **Ayudas para emprendimiento tecnológico:** como el programa de JCYL Emprende que ofrece ayudas de hasta 10.000€ a jóvenes emprendedores.
- **Subvenciones para digitalización.**
- **Campaña de crowdfunding del proyecto o micromecenazgo.**
- **Programas de apoyo a startups o programas lanzadera.**
- **Hackathons con premio en metálico:** algunos son el Hackathon de Telefónica (premios de hasta 6.000€), el Samsung Dev Spain (premios en dispositivos y en metálico) y los hackathons de la comunidad de desarrolladores de Google.

Planificación temporal (cronograma):

Semanas 1-2:

- Investigación y planificación.
- Diseño inicial.
- Configuración del repositorio.

Semanas 3-6:

- Desarrollo del backend.
- Base de datos.
- Sistema de autenticación.

Semanas 7-11:

- Desarrollo frontend.
- Gestión de salas.
- Integración API REST.

Semanas 12-15:

- Implementación de lógica de juego.
- WebSockets y tiempo real.

Semanas 16-18:

- Pruebas y corrección de errores.
- Optimización visual.

Semanas 19-20:

- Documentación.
- Despliegue.
- Preparación de presentación final.

4. Tecnologías escogidas y justificación.

Frontend

- **HTML:** Lenguaje de marcado encargado de estructurar el contenido de las distintas vistas de la aplicación y es el estándar universal. Compatible con todos los navegadores.
- **CSS:** Permite controlar el diseño visual sin afectar a la lógica. Mejora la experiencia del usuario. Ayuda a que el juego sea intuitivo y atractivo.
- **Bootstrap (5.3.3):** Ayuda a crear una web responsive, con componentes reutilizables y reduce el tiempo de desarrollo. Se puede complementar con el CSS normal.
- **Sass (1.99.0):** Preprocesador CSS que facilita el uso de variables, mixins y anidación.
- **Vue.js (3.5.28):** Tiene componentes reutilizables y mejora la organización del código. Permite actualizar la interfaz automáticamente cuando cambia el estado del juego.
- **Vue Router (5.0.2):** Navegación SPA sin recarga de página.
- **Vuex (4.1.0):** Gestión del estado global (sala, jugador, fase, notificaciones).
- **Axios (1.6.0):** Cliente HTTP para consumo de la API REST.
- **STOMP + SockJS (7.3.0 / 1.6.1):** Implementación de comunicación en tiempo real mediante WebSockets.
- **Vite (7.3.1):** Build tool rápido y servidor de desarrollo.

Backend

- **Java (21):** Lenguaje orientado a objetos, de tipado estático y ampliamente utilizado en entornos empresariales.
- **Spring Boot (4.0.3):** Framework que reduce código repetitivo, configuración rápida e integración con ecosistema Spring.
- **Spring Data JPA / Hibernate:** ORM que evita escribir SQL manual, mapeo objeto-relacional.
- **Spring Web (REST):** Creación de endpoints API REST.
- **Spring WebSockets:** Comunicación en tiempo real (actualización de fase día/noche, muertes).
- **Spring Security + JWT:** Autenticación stateless segura.
- **Spring Validation:** Validación de datos de entrada.
- **Lombok (1.18.38):** Generación automática de getters, setters, constructores.
- **MySQL Connector:** Conexión a base de datos MySQL.

Base de datos

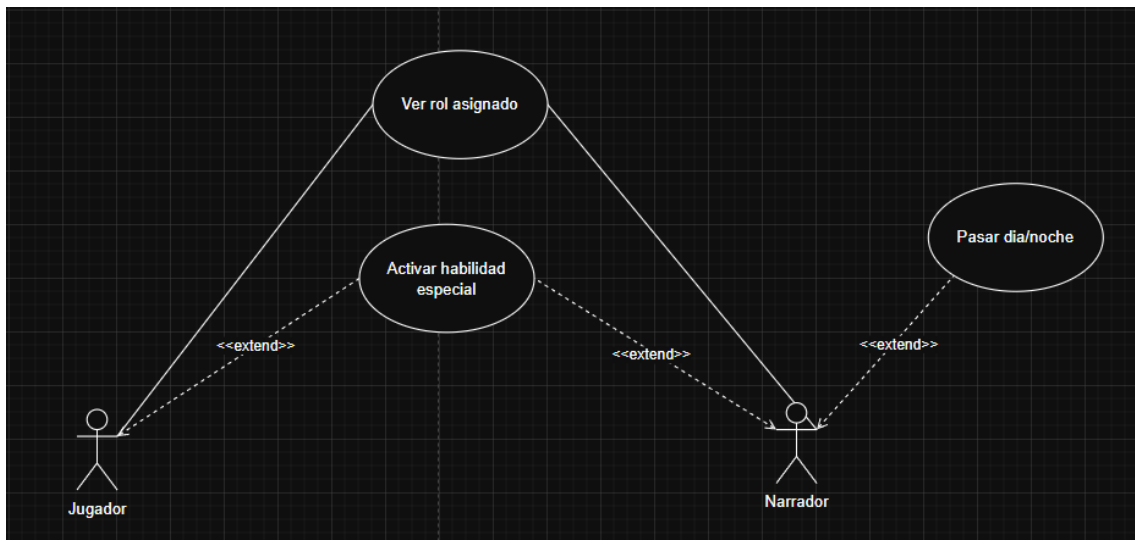
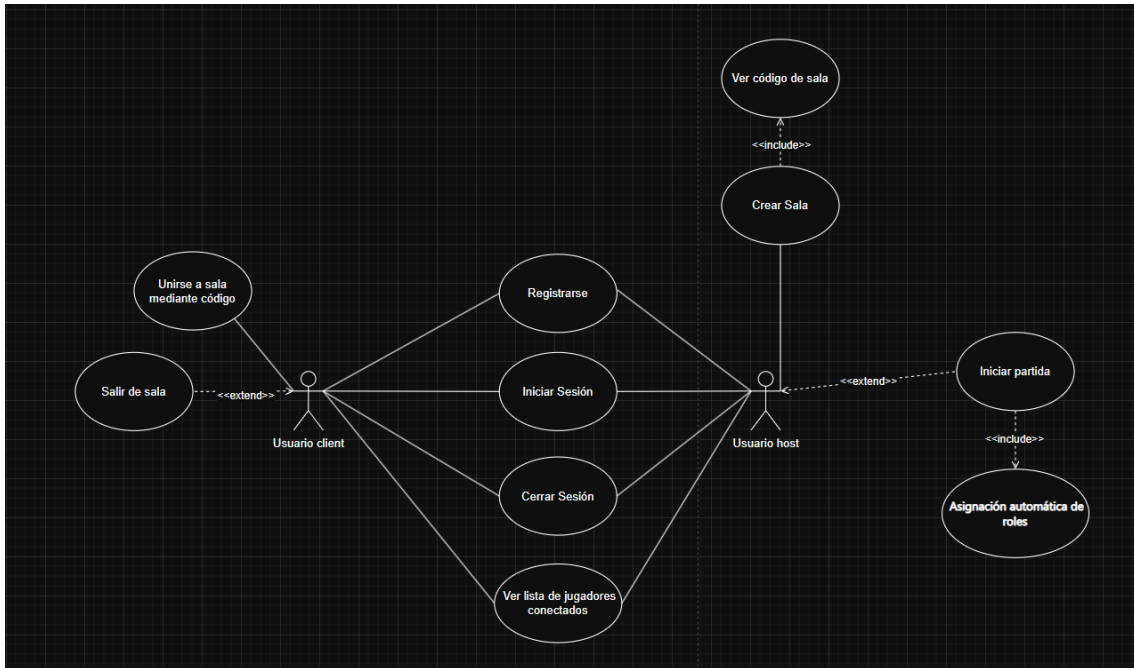
- **MySQL:** ayuda a generar bases de datos estables, estructuradas y que se integran muy fácilmente con otras tecnologías. Además posee un amplio soporte y documentación.

Otras

- **Docker:** facilita el despliegue y la pruebas en distintos dispositivos. Ideal para presentar y evaluar el proyecto. Permite ejecutar backend y base de datos fácilmente.
- **Git:** permite un control de versiones y del flujo de trabajo. Mejora la comunicación entre los distintos miembros del proyecto y posee un historial de cambios.
- **JUnit + Mockito:** Testing backend.
- **Vitest (2.1.8) + Vue Test Utils:** Testing unitario frontend.
- **Adobe Illustrator:** software de diseño gráfico vectorial para la creación del logo y montaje del footer.
- **Adobe Photoshop:** software de edición fotográfica utilizado para la edición y retoque de imágenes.
- **Artist:** Plataforma utilizada para la generación de recursos multimedia e ilustraciones basadas en inteligencia artificial a partir de prompts definidos por el equipo.

5. Diseño de la aplicación.

5.1. Diagramas y definición de casos de uso



➡ CU01 – Registrarse

- **Actor principal:** Usuario
- **Actores secundarios:** Sistema de Autenticación
- **Objetivo:** permitir que un nuevo usuario cree una cuenta para poder jugar o crear salas.
- **Precondiciones:** el usuario no debe estar registrado previamente.
- **Postcondiciones**
 - Se crea una cuenta con usuario y contraseña.
 - El usuario queda registrado en la base de datos.
- **Flujo principal**
 - El usuario abre la pantalla de registro.
 - Introduce nombre, contraseña y confirmación.
 - Pulsa “Crear cuenta”.
 - El sistema valida el formato de los datos.
 - El sistema verifica que el usuario no existe.
 - El sistema guarda la cuenta.
 - El usuario recibe un mensaje de éxito.
- **Alternativos**
 - **A1** – Usuario ya registrado → Mostrar error.
 - **A2** – Campos vacíos o inválidos → Mostrar advertencia.
 - **A3** – Fallo en BBDD → Opción de reintentar.

➡ CU02 – Crear sala estándar

- **Actor principal:** Gestor de Salas
- **Secundarios:** Narrador
- **Objetivo:** crear sala con reglas predefinidas.
- **Precondiciones**
 - Sesión iniciada.
 - Usuario no debe estar en ninguna otra sala.
- **Postcondiciones**
 - Sala creada y el usuario es el narrador.
 - Código de sala generado.
- **Flujo principal**
 - Click en “Crear sala (estándar)”.
 - Sistema genera sala.
 - Asigna configuración por defecto.
 - Muestra código de sala.
 - El narrador accede a la sala.
- **Alternativos**
 - **A1** – Error creando sala.

➡ CU03 – Salir de sala

- **Actor principal:** Jugador / Narrador
- **Objetivo:** abandonar la sala antes del inicio de partida.
- **Precondiciones:** usuario dentro de la sala (lobby).
- **Postcondiciones**
 - Usuario vuelve al menú principal.
 - La sala se actualiza (si era el narrador, la sala se elimina).
- **Flujo principal**
 - Usuario pulsa “Salir”.
 - Sistema pide confirmación.
 - Usuario confirma.
 - Sistema lo retira de la sala.
 - Vuelve al menú principal.
- **Alternativos**
 - **A1** – Usuario cancela salida.

➡ CU04 – Iniciar sesión

- **Actor principal:** Usuario
- **Secundarios:** Sistema de Autenticación
- **Objetivo:** acceder al sistema usando credenciales válidas.
- **Precondiciones:** usuario registrado.
- **Postcondiciones**
 - Sesión iniciada.
 - Usuario accede a la pantalla principal.
- **Flujo principal**
 - Usuario introduce nombre y contraseña.
 - Sistema valida credenciales.
 - Si son correctas, inicia sesión.
 - Se redirige a pantalla principal (Crear sala / Unirse a sala).
- **Alternativos**
 - **A1** – Contraseña incorrecta → Aviso.
 - **A2** – Usuario inexistente.

➡ CU05 – Cerrar sesión

- **Actor principal:** Usuario
- **Objetivo:** salir del sistema de forma segura.
- **Precondiciones:** usuario con sesión activa.
- **Postcondiciones**
 - Sesión destruida.
 - Retorna a pantalla de login.
- **Flujo principal**
 - Usuario pulsa “Cerrar sesión”.
 - Sistema elimina la sesión.
 - Redirección al login.

➡ CU06 – Ver código de sala

- **Actor principal:** Creador Sala
- **Objetivo:** obtener el código de la sala para compartirlo.
- **Precondiciones:** narrador (es el creador por defecto) dentro de una sala.
- **Postcondiciones:** se muestra el código.
- **Flujo principal**
 - El creador abre la pantalla de la sala.
 - Sistema muestra código visible.
 - Creador lo copia al portapapeles.

➡ CU07 – Cerrar sala

- **Actor principal:** Narrador
- **Secundarios:** Gestor de Salas
- **Objetivo:** Finalizar y destruir la sala manualmente.
- **Precondiciones**
 - La sala debe estar activa.
 - Narrador en control.
- **Postcondiciones**
 - Todos los jugadores son expulsados.
 - Sala eliminada.
 - Se vuelve al menú principal.
- **Flujo principal**
 - Narrador pulsa “Cerrar sala”.
 - Sistema pide confirmación.
 - Narrador confirma.
 - Sistema elimina sala.
 - Jugadores vuelven a inicio.
 - Narrador vuelve al menú principal.
- **Alternativos**
 - **A1** – Narrador cancela cierre.

➡ CU08 – Unirse a sala mediante código

- **Actor principal:** Jugador
- **Secundarios:** Gestor de Salas
- **Objetivo:** permitir que un jugador acceda a una sala existente.
- **Precondiciones**
 - Jugador autenticado.
 - Debe tener un nombre en el sistema.
- **Postcondiciones**
 - Jugador aparece en la lista de la sala como “conectado”.
 - La sala incrementa el contador de jugadores.
- **Flujo principal**
 - Jugador introduce código de sala.
 - Pulsa “Unirse”.
 - El sistema verifica que la sala existe.
 - El sistema verifica que la partida no haya iniciado.
 - El jugador se agrega a la lista de la sala.
 - Se muestra la pantalla de espera.
- **Alternativos**
 - **A1** – Código inexistente
 - **A2** – Sala llena
 - **A3** – La partida ya comenzó → se deniega la entrada

➡ CU09 – Iniciar partida

- **Actor principal:** Narrador
- **Secundarios:** Motor de Roles, Motor de Fase
- **Objetivo:** iniciar oficialmente la partida.
- **Precondiciones**
 - Mínimo de jugadores alcanzado.
 - Todos están conectados.
 - Configuración válida.
- **Postcondiciones**
 - Roles asignados.
- **Flujo principal**
 - Narrador pulsa “Iniciar partida”.
 - Sistema verifica requisitos.
 - Motor de Roles asigna roles.
 - Se notifica a cada jugador su rol.

- **Alternativos**
 - **A1** – Jugadores desconectados.
 - **A2** – No hay jugadores suficientes.
 - **A3** – Configuración inválida.

➡ **CU10 – Asignación automática de roles**

- **Actor principal:** Motor de Roles
- **Actores involucrados:** Narrador (inicia la acción), Jugadores
- **Objetivo:** asignar roles de forma automática basada en la configuración de la sala y el número de jugadores.
- **Precondiciones**
 - Partida no iniciada.
 - Mínimo número de jugadores alcanzado.
 - Configuración válida de roles.
- **Postcondiciones**
 - Cada jugador recibe un rol oculto.
- **Flujo principal**
 - Narrador pulsa “Iniciar partida”.
 - El Motor de Roles calcula lobos.
 - Calcula distribución restante de roles.
 - Asigna roles a cada jugador de forma aleatoria.
 - Envía roles a cada jugador.
 - Actualiza estado de partida.
- **Alternativos**
 - **A1** – Roles insuficientes para este número de jugadores → aviso.
 - **A2** – Error al asignar roles → reintentar.

➡ CU11 – Mostrar sala de espera

- **Actor principal:** Jugador / Narrador
- **Objetivo:** mostrar la sala antes de iniciar la partida con la lista de jugadores conectados.
- **Precondiciones**
 - Sala creada.
 - Partida no iniciada.
- **Postcondiciones:** lista actualizada en tiempo real.
- **Flujo principal**
 - Usuario accede a la sala.
 - Sistema muestra lista de jugadores conectados.
 - Usuarios ven cuando alguien se une o sale.
- **Alternativos**
 - **A1** – Jugador se desconecta → actualizar lista.
 - **A2** – Narrador cierra sala → redirigir a todos.

➡ CU12 – Ver rol asignado

- **Actor principal:** Jugador
- **Objetivo:** permitir que cada jugador vea su rol secreto.
- **Precondiciones**
 - Partida iniciada.
 - Motor de roles completó asignación.
- **Postcondiciones**
 - El jugador conoce su rol.
 - No puede cambiarlo ni mostrarlo a otros desde la interfaz.
- **Flujo principal**
 - El jugador recibe notificación: “Tu rol ha sido asignado”.
 - Pulsa el botón “Ver mi rol”.
 - El sistema muestra su rol (lobo, aldeano, rol especial...).
 - Jugador cierra la ventana de rol.
- **Alternativos**
 - **A1** – Jugador pierde conexión → se mostrará al reconectar.

➡ CU13 – Determinar condición de victoria

- **Actor principal:** Motor del Juego
- **Actores secundarios:** Narrador, Jugadores
- **Objetivo:** calcular automáticamente si el juego debe terminar por victoria de aldeanos o lobos.
- **Precondiciones**
 - Al finalizar un día o una noche.
 - Debe existir una nueva distribución de jugadores vivos.
- **Postcondiciones**
 - El sistema declara unos vencedores y finaliza la partida.
- **Condiciones típicas:**
 - Ganan lobos: 0 aldeanos vivos.
 - Ganan aldeanos: 0 lobos vivos.
 - Ganan Enamorados: solo ellos dos siguen vivos.
 - Empate: situación excepcional en la que ningún jugador permanece con vida.
- **Flujo principal**
 - Sistema evalúa el número de lobos y aldeanos vivos.
 - Comprueba condiciones de victoria.
 - Si se cumple alguna, se declara ganador.
 - Muestra pantalla de resultados.
 - El narrador y los jugadores regresan al menú principal.
- **Alternativos**
 - **A1** – Nadie ha ganado aún → continuar la partida.

➡ CU14– Finalizar partida

- **Actor principal:** Narrador
- **Actores secundarios:** Motor del Juego
- **Objetivo:** cerrar la partida manualmente, incluso si no se cumplen condiciones automáticas.
- **Precondiciones:** Narrador debe estar dentro de una partida activa.
- **Postcondiciones**
 - Partida finalizada.
 - Todos los jugadores son llevados a la pantalla de resultados.
 - La sala se destruye.
- **Flujo principal**
 - Narrador pulsa botón “Finalizar partida”.
 - Sistema solicita confirmación.
 - Narrador confirma.
 - Sistema finaliza partida.
 - Muestra resultados y roles finales.
 - Todos los jugadores regresan al menú principal.
- **Alternativos**
 - **A1** – Narrador cancela el cierre.

➡ CU15 – Activar habilidad especial

- **Ejemplos:** bruja, vidente, niña, cazador...
- **Actor principal:** Jugador con rol especial
- **Secundarios:** Narrador, Motor de Fase
- **Objetivo:** permitir a ciertos roles activar sus habilidades en momentos específicos.
- **Precondiciones**
 - La partida debe estar en curso.
 - La fase debe permitir la habilidad (ej. bruja → noche).
 - El jugador debe estar vivo, salvo en habilidades activables tras la muerte.
- **Postcondiciones**
 - Se registra el uso de la habilidad.
 - La habilidad puede quedar gastada (si es de uso limitado).
 - El narrador puede ver las acciones en un panel privado.
- **Flujo principal**
 - El sistema detecta que es el turno del rol (ej. bruja).
 - El jugador recibe una notificación: “Puedes usar tu habilidad”.
 - El jugador abre el panel de habilidad.
 - Selecciona si desea usarla.
 - Selecciona objetivo (si aplica).
 - El sistema procesa la acción.
- **Alternativos**
 - **A1** – El jugador no actúa → habilidad omitida.
 - **A2** – El jugador ya usó la habilidad y no tiene más usos.
 - **A3** – El jugador muere antes de actuar.

➡ CU16 – Devorar (Hombre Lobo)

- **Actor principal:** Hombre Lobo
- **Secundarios:** Motor de Fase, Narrador
- **Objetivo:** permitir al hombre lobo elegir una víctima para devorar durante la noche.
- **Precondiciones**
 - Ser hombre lobo.
 - Estar en fase de noche.
 - Estar vivo.
 - No haber devorado esta noche (uso único).
- **Postcondiciones**
 - Víctima marcada para morir.
 - Narrador recibe notificación privada.
- **Flujo principal**
 - Sistema activa turno de hombre lobo.
 - Jugador abre panel de devorar.
 - Selecciona víctima de la lista de jugadores vivos.
 - Confirma la selección.
 - Sistema registra la víctima.
 - Notifica al narrador.
- **Alternativos**
 - **A1** – No hay hombres lobo vivos → omitir.
 - **A2** – El hombre lobo no selecciona → sin víctima.
 - **A3** – Selecciona a otro lobo vivo → error, no pueden elegirse entre lobos.

➡ CU17 – Ver rol (Vidente)

- **Actor principal:** Vidente
- **Secundarios:** Motor de Fase, Narrador
- **Objetivo:** permitir a la vidente ver el rol de otro jugador durante la noche.
- **Precondiciones**
 - Ser vidente.
 - Estar en fase de noche.
 - Estar vivo.
 - No haber usado su habilidad esta noche (o tenerla disponible).
- **Postcondiciones**
 - Vidente conoce el rol del objetivo.
 - Narrador recibe notificación del resultado.
- **Flujo principal**
 - Sistema activa turno de vidente.
 - Jugador abre panel de videncia.
 - Selecciona un jugador para ver su carta.
 - Confirma la selección.
 - Sistema revela el rol (aldeano/lobo/especial).
 - Muestra resultado al jugador.
- **Alternativos**
 - **A1** – Vidente no usa habilidad → omitir.
 - **A2** – Ya usó su habilidad esta noche → no puede usar.
 - **A3** – No hay jugadores vivos restantes → omitir.

➡ CU18 – Usar poción (Bruja)

- **Actor principal:** Bruja
- **Secundarios:** Motor de Fase, Narrador
- **Objetivo:** permitir a la bruja usar sus pociones de vida o muerte durante la noche.
- **Precondiciones**
 - Ser bruja.
 - Estar en fase de noche.
 - Estar viva.
 - En caso de estar marcada para morir, puede utilizar la poción de vida sobre sí misma.
 - Tener pociones disponibles (x1 vida, x1 muerte).
- **Postcondiciones**
 - Se aplica el efecto correspondiente.
 - Se marca la poción como usada.
- **Flujo principal**
 - Sistema activa turno de bruja.
 - Jugador recibe notificación.
 - Elige usar poción de vida, muerte, ambas o ninguna.
 - Selecciona objetivo(s) si aplica.
 - Confirma decisiones.
 - Sistema aplica efectos.
- **Alternativos**
 - **A1** – Bruja no usa pociones → turno terminado.
 - **A2** – Poción de vida ya usada → solo puede usar muerte.
 - **A3** – Poción de muerte ya usada → solo puede usar vida.

➡ **CU19 – Espiar (Niña)**

- **Actor principal:** Niña
- **Secundarios:** Motor de Fase, Narrador
- **Objetivo:** permitir a la niña espiar a los lobos durante la noche.
- **Precondiciones**
 - Ser niña.
 - Estar en fase de noche.
 - Estar viva.
- **Postcondiciones:** Niña ve entre un grupo reducido de jugadores la probabilidad de que uno de ellos sea un lobo.
- **Flujo principal**
 - Sistema activa turno de niña.
 - Jugador abre panel de espionaje.
 - Selecciona el botón para “espiar desde la ventana”.
 - Confirma la selección.
 - Sistema revela un grupo reducido de jugadores sin revelar su rol.
 - La niña ve que entre ese grupo, uno es probable que sea un lobo.
 - Niña pulsa en el botón de volverse a dormir.
- **Alternativos**
 - **A1** – Niña no espía → omitir.
 - **A2** – Error al observar → mostrar error.

➡ CU20 – Enamorar (Cupido)

- **Actor principal:** Cupido
- **Secundarios:** Motor de Fase, Narrador
- **Objetivo:** permitir a Cupido elegir a dos jugadores para que se enamoren.
- **Precondiciones**
 - Ser cupido.
 - No haber utilizado su habilidad previamente.
 - Estar vivo.
 - Puede elegirse a él mismo como uno de los enamorados
- **Postcondiciones**
 - Ambos jugadores quedan marcados como enamorados.
 - Se crea un nuevo bando (enamorados).
- **Flujo principal**
 - Sistema activa turno de cupido (Noche 1).
 - Jugador abre panel de cupido.
 - Selecciona al primer jugador.
 - Selecciona el segundo jugador.
 - Confirma el emparejamiento.
 - Sistema marca a ambos como enamorados.
 - Les notifica su nueva condición.
- **Alternativos**
 - **A1** – Cupido no enamora → omitir.
 - **A2** – Selecciona el mismo jugador → error.

➡ CU21 – Disparar (Cazador)

- **Actor principal:** Cazador
- **Secundarios:** Motor de Fase
- **Objetivo:** permitir al cazador elegir a una víctima al morir.
- **Precondiciones**
 - Ser cazador.
 - Estar marcado para morir.
 - No haber disparado aún.
- **Postcondiciones**
 - Víctima marcada para morir queda eliminada de la partida.
 - Cazador queda marcado como muerto.
- **Flujo principal**
 - Sistema detecta muerte del cazador.
 - Muestra panel de disparo al cazador.
 - Jugador selecciona una víctima.
 - Confirma la selección.
 - Sistema aplica muerte.
 - Notifica a todos.
- **Alternativos**
 - **A1** – Cazador no selecciona → sin víctimas adicionales.
 - **A2** – Selecciona jugador ya muerto → no permitido.

➡ CU22 – Elegir mentor (Niño Salvaje)

- **Actor principal:** Niño Salvaje
- **Secundarios:** Motor de Fase, Narrador
- **Objetivo:** permitir al niño salvaje elegir a un mentor durante la noche.
- **Precondiciones**
 - Ser niño salvaje.
 - No haber elegido mentor aún.
 - Estar vivo.
- **Postcondiciones**
 - Jugador seleccionado queda marcado como mentor del niño.
 - Si el mentor muere, el niño se convierte en lobo.
- **Flujo principal**
 - Sistema activa turno de niño salvaje.
 - Jugador abre panel.
 - Selecciona un jugador como mentor.
 - Confirma la selección.
 - Sistema registra al mentor.

- **Alternativos**
 - **A1** – Niño no elige mentor → omitir.
 - **A2** – Niño ya eligió → no puede cambiar.

➡ **CU23 – Pasar de día a noche**

- **Actor principal:** Narrador
- **Secundarios:** Motor de Fase
- **Objetivo:** cambiar el estado del juego para permitir acciones nocturnas.
- **Precondiciones**
 - Partida en curso.
 - Narrador debe decidir el cambio de fase.
- **Postcondiciones**
 - Estado pasa a Noche.
 - Lobos y personajes con habilidades nocturnas pueden actuar.
 - Se muestran mensajes automáticos a todos los jugadores.
- **Flujo principal**
 - Narrador pulsa en el botón de noche.
 - Sistema bloquea acciones de día.
 - Sistema activa la fase nocturna.
 - Se notifican a los jugadores.
- **Alternativos**
 - **A1** – El narrador cancela el cambio de fase.

➡ CU24 – Pasar de noche a día

- **Actor principal:** Narrador
- **Actores secundarios:** Motor de Fase, Jugadores
- **Objetivo:** cambiar la fase del juego para resolver las acciones nocturnas y mostrar resultados.
- **Precondiciones**
 - El juego debe estar en fase de noche.
 - El narrador debe haber finalizado la supervisión de acciones nocturnas.
- **Postcondiciones**
 - Se resuelven muertes y eventos nocturnos.
 - Se activa la fase de día.
- **Flujo principal**
 - El narrador pulsa el botón para que se haga de día.
 - El sistema cierra las acciones nocturnas.
 - El sistema determina muertes y eventos.
 - Muestra mensaje de aviso de que vuelve a ser de día.
 - Se anuncian los jugadores eliminados (según reglas).
- **Alternativos**
- **A1** – Narrador cancela el proceso.
- **A2** – No hay acciones nocturnas → pasar directamente al día.

➡ CU25 – Elegir alcalde/ alguacil

- **Actor principal:** Narrador/ Jugadores
- **Secundarios:** Motor de Fase, Motor de Votación
- **Objetivo:** permitir la elección de un alcalde mediante votación.
- **Precondiciones**
 - Partida en curso.
 - Narrador habilita elección de alcalde.
 - No hay alcalde activo.
- **Postcondiciones**
 - Un jugador queda marcado como alcalde.
 - El alcalde tiene voto de calidad en caso de empate.
- **Flujo principal**
 - Narrador activa “Elegir alcalde”.
 - Sistema inicia la votación.
 - Cada jugador vivo selecciona un candidato.
 - El más votado se convierte en alcalde.
 - En caso de empate, se volverán a realizar las votaciones.

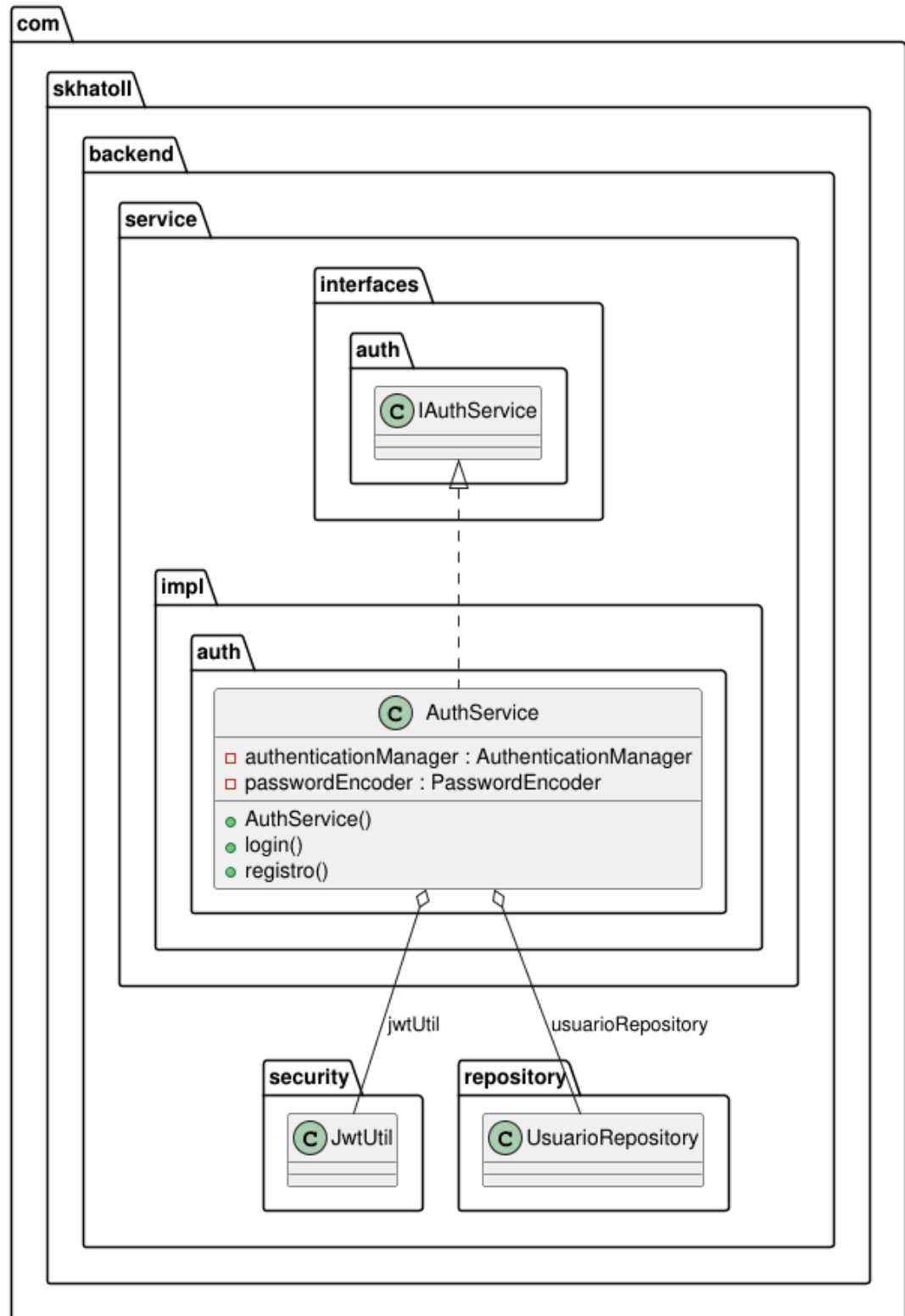
- **Alternativos**
 - **A1** – Un solo candidato → se nombra directamente.
 - **A2** – Empate en votos → reiniciar votaciones.
 - **A3** – Narrador cancela → no hay alcalde

➡ **CU26 – Votar para linchar**

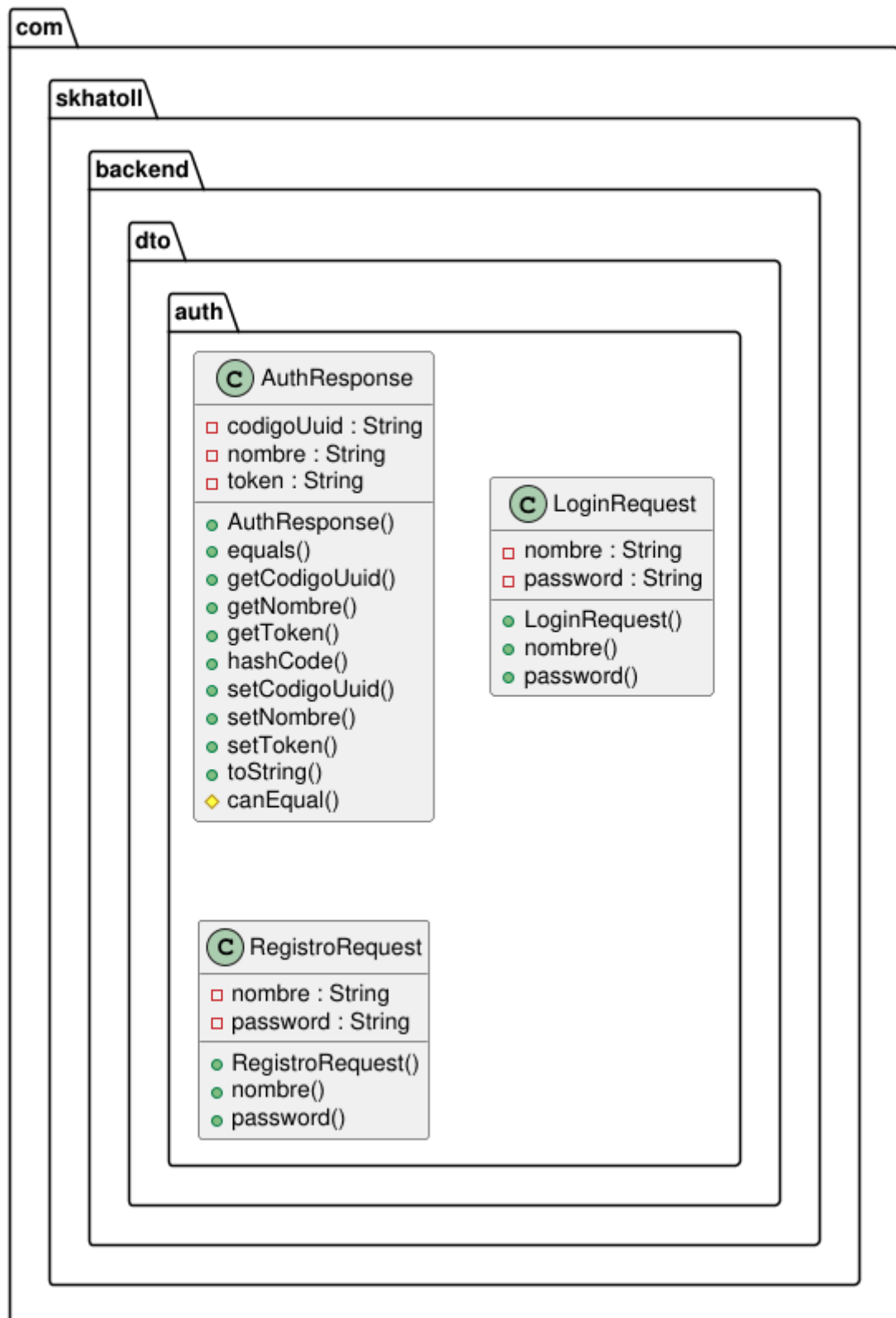
- **Actor principal:** Jugadores
- **Secundarios:** Motor de Fase, Motor de Votación
- **Objetivo:** permitir a los jugadores votar para eliminar a un sospechoso.
- **Precondiciones**
 - Partida en fase de día.
 - Narrador habilita votación.
 - Al menos un jugador vivo.
- **Postcondiciones**
 - El jugador con más votos es linchado.
 - Si hay alcalde y decide usar voto de calidad, ese voto cuenta doble.
 - Si hay empate el alcalde decide quién es el jugador eliminado.
- **Flujo principal**
 - Narrador inicia “Votación para linchar”.
 - Cada jugador vivo vota por otro jugador vivo.
 - Sistema cuenta votos.
 - El jugador con más votos es eliminado.
 - En caso de empate, el alcalde decide.
- **Alternativos**
 - **A1** – Todos votan por sí mismos → no muere nadie.
 - **A2** – Jugador no vota → voto en blanco.
 - **A3** – Alcalde usa voto de calidad → cuenta como 2 votos.

5.2. Diagramas de clase

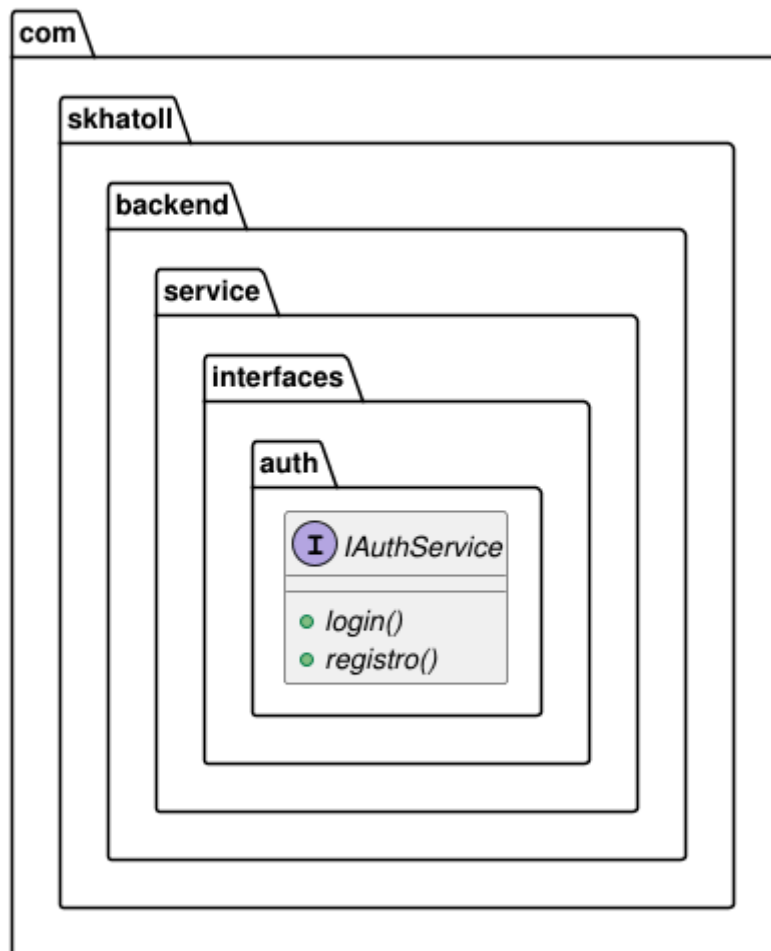
AUTH's Class Diagram



AUTH's Class Diagram

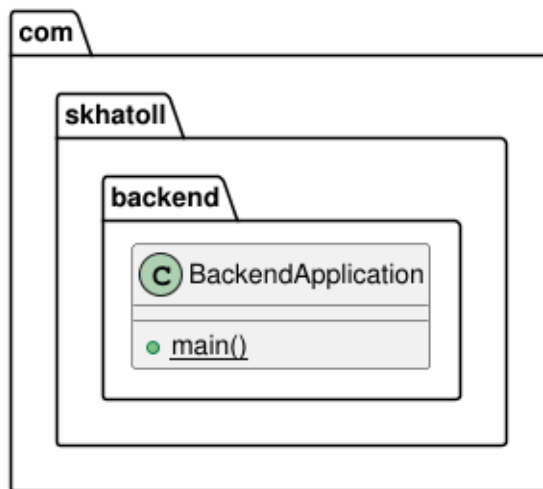


AUTH's Class Diagram



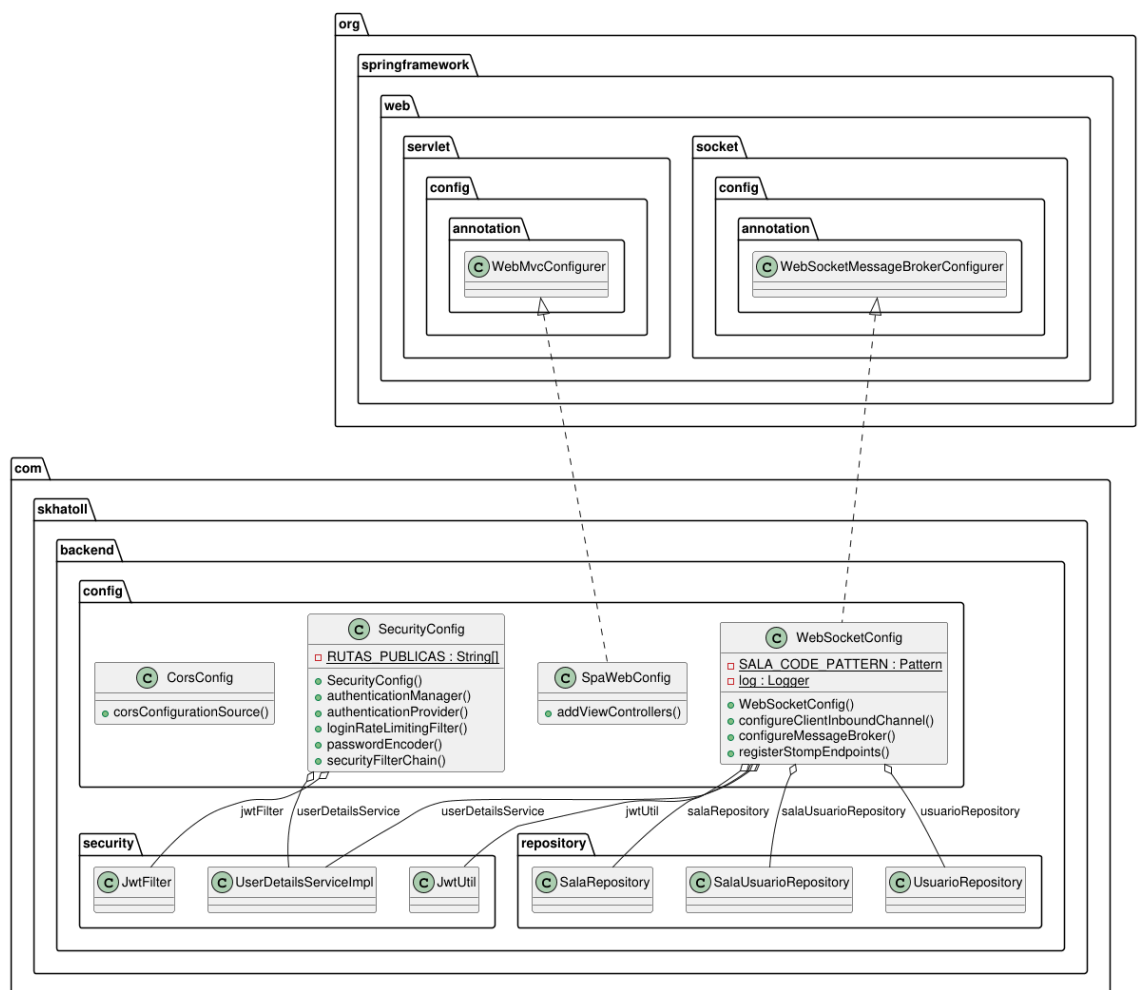
PlantUML diagram generated by SketchIt! (<https://bitbucket.org/pmameur/sketch.it>)
For more information about this tool, please contact philippe.mesmeur@gmail.com

BACKEND's Class Diagram



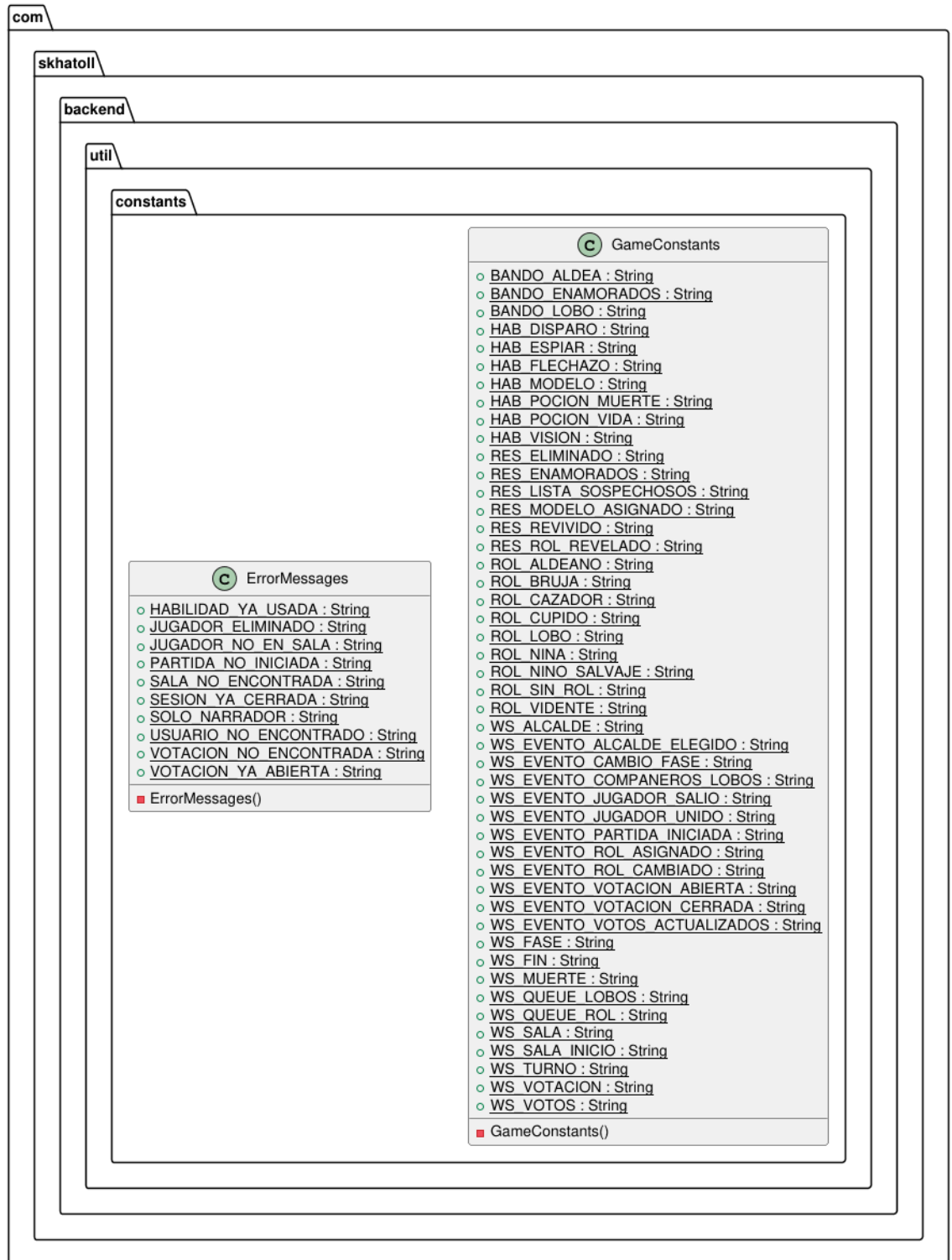
PlantUML diagram generated by SketchIt! (<https://bitbucket.org/pmesmeur/sketch.it>)
For more information about this tool, please contact philippe.mesmeur@gmail.com

CONFIG's Class Diagram

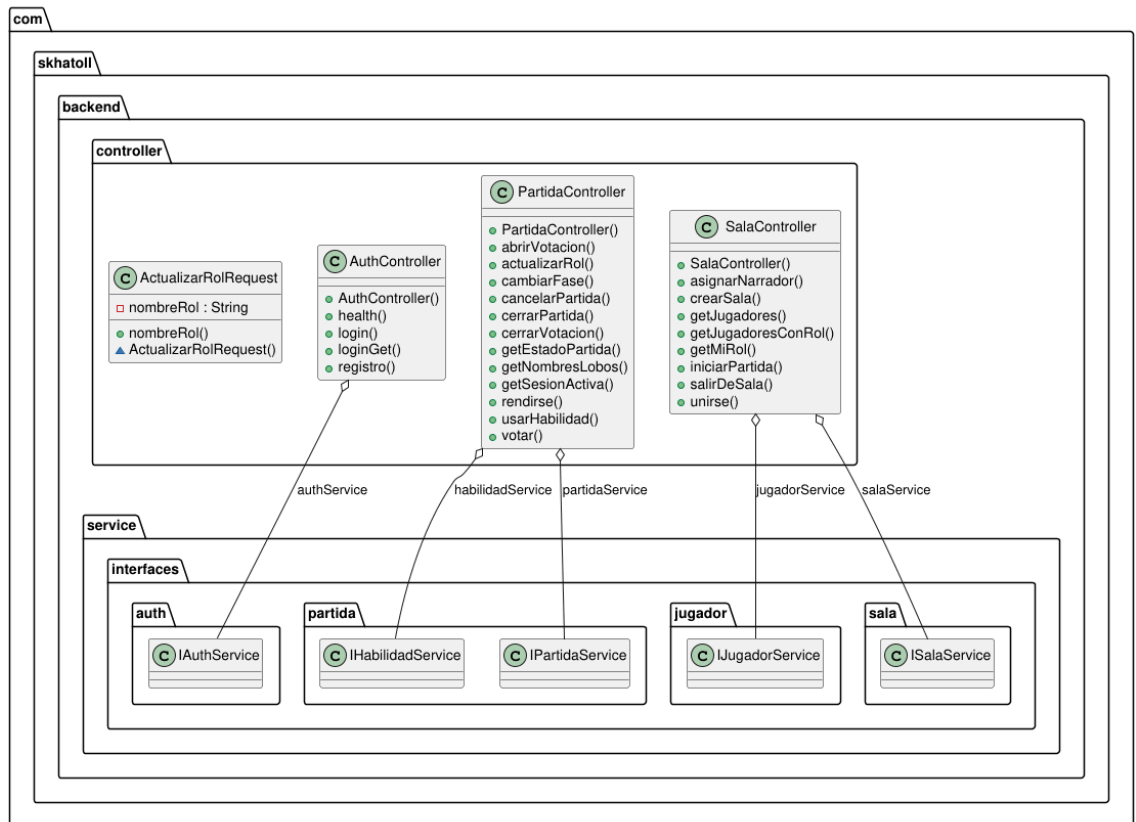


PlantUML diagram generated by SketchIt! (<https://bitbucket.org/pmesmeur/sketch.it>)
For more information about this tool, please contact philippe.mesmeur@gmail.com

CONSTANTS's Class Diagram



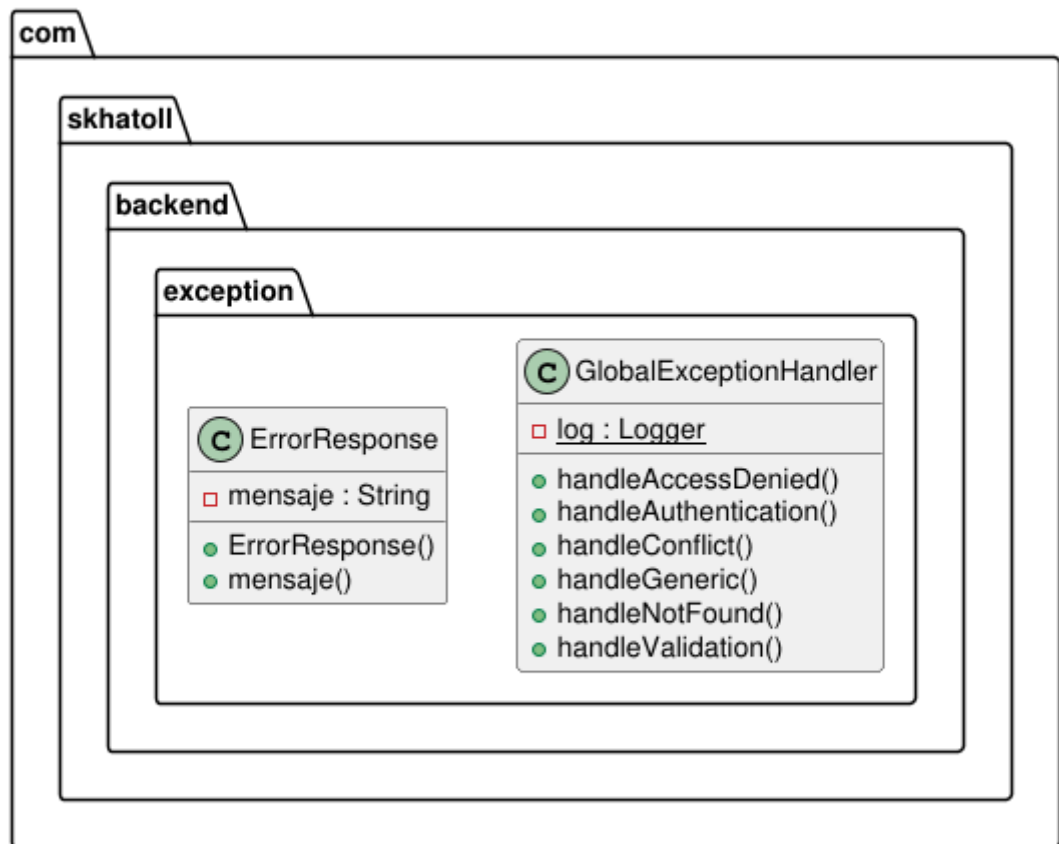
CONTROLLER's Class Diagram



PlantUML diagram generated by SketchIT! (<https://bitbucket.org/pmesmeur/sketch.it/>)
For more information about this tool, please contact philippe.mesmeur@gmail.com

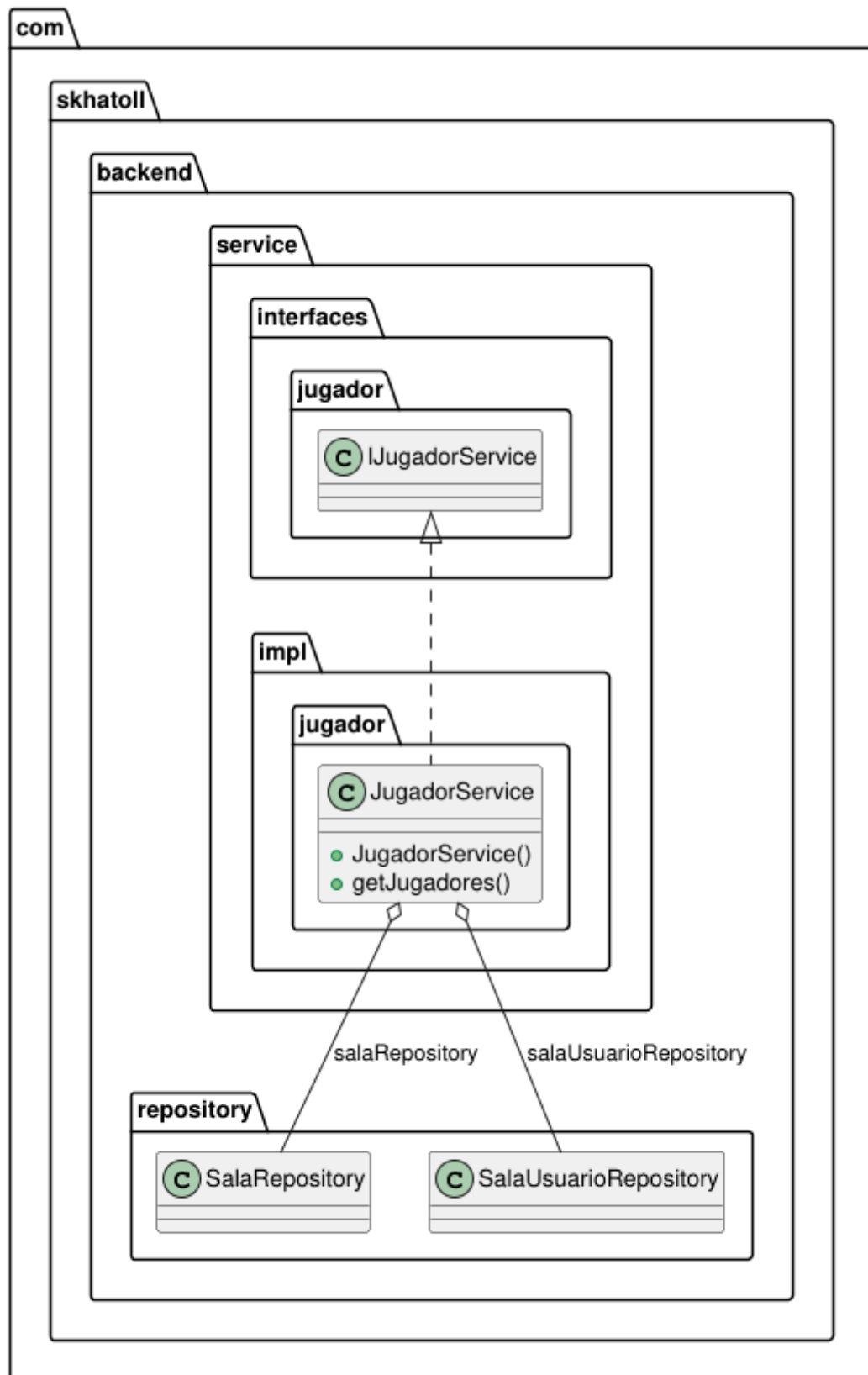


EXCEPTION's Class Diagram

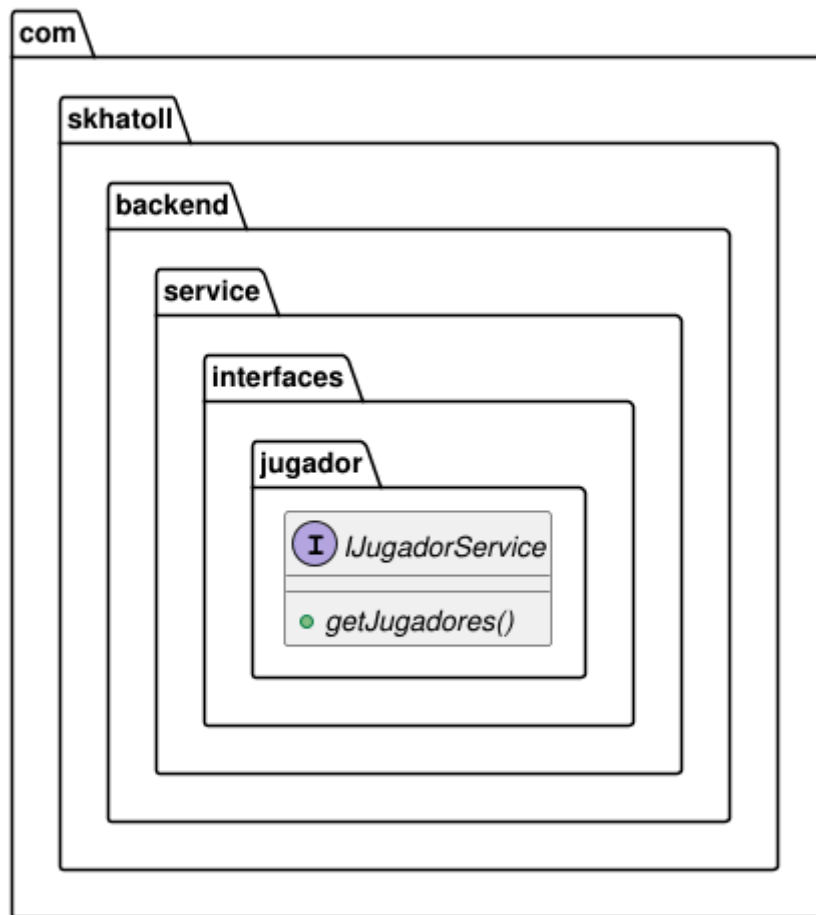


PlantUML diagram generated by SketchIt! (<https://bitbucket.org/pmesmeur/sketch.it>)
For more information about this tool, please contact philippe.mesmeur@gmail.com

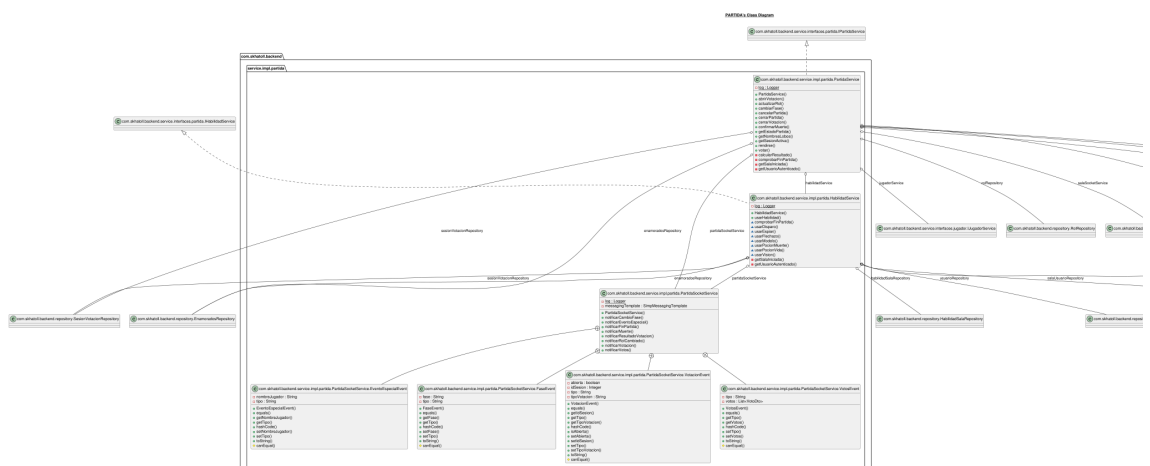
JUGADOR's Class Diagram



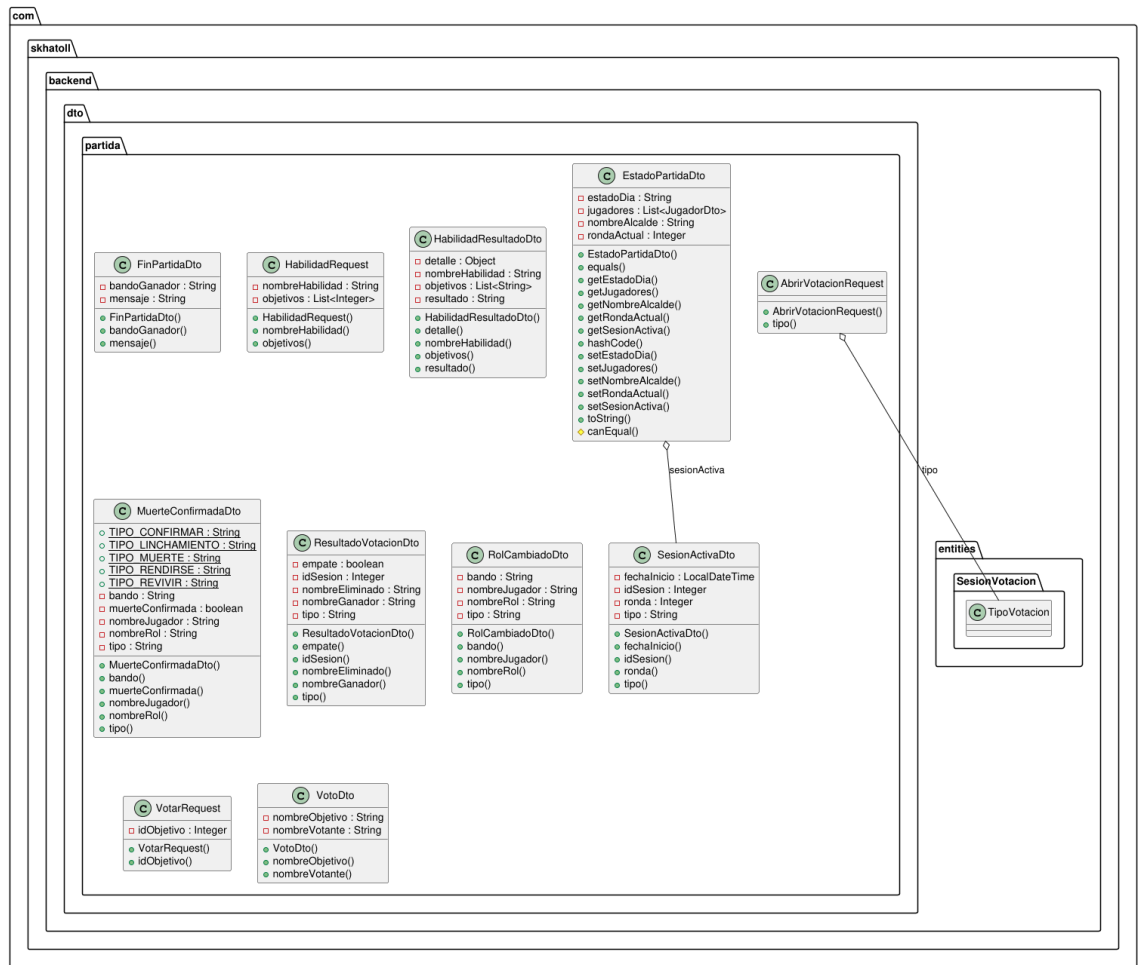
JUGADOR's Class Diagram



PlantUML diagram generated by SketchIt! (<https://bitbucket.org/pmesmeur/sketch.it>)
For more information about this tool, please contact philippe.mesmeur@gmail.com.

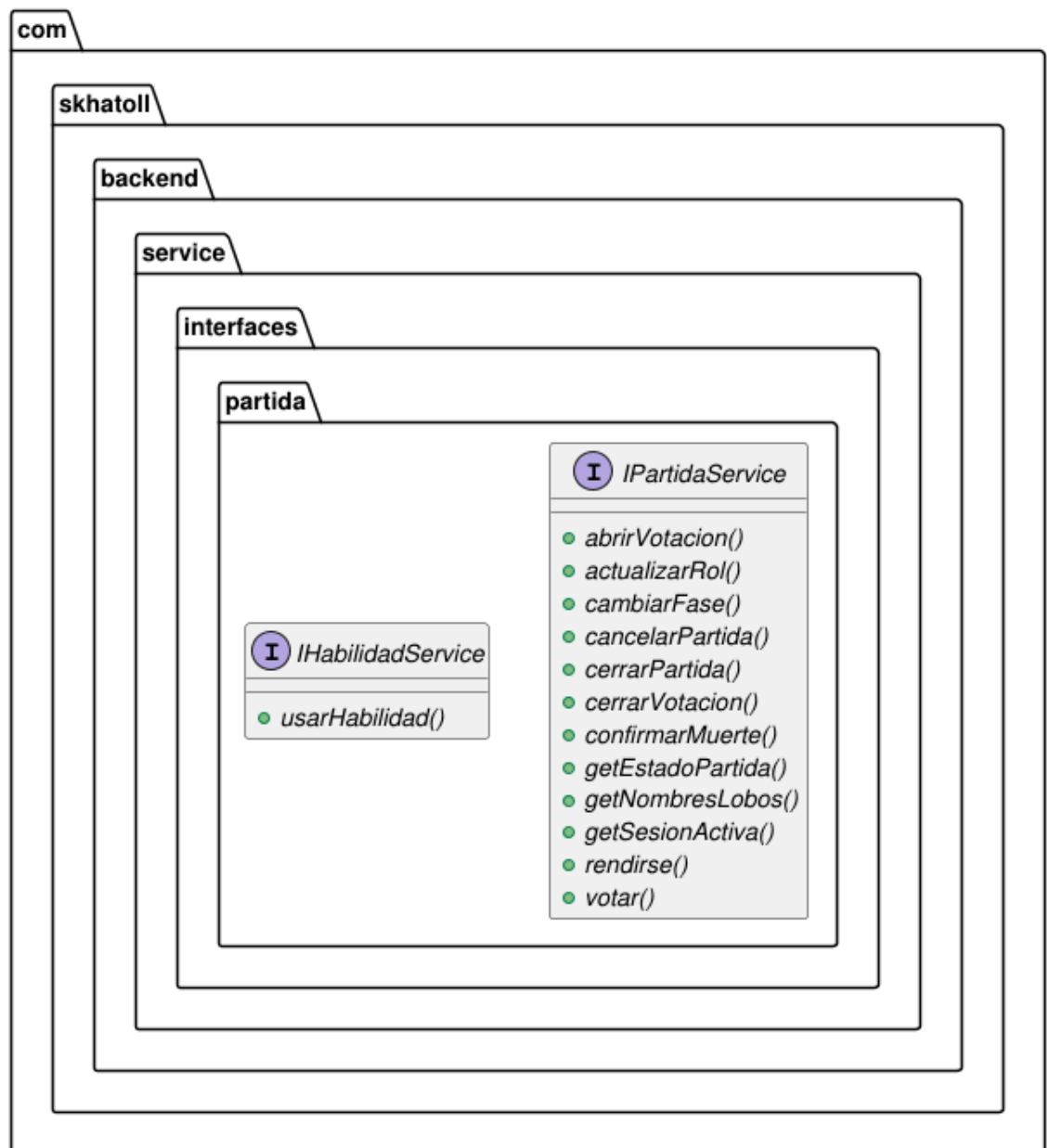


PARTIDA's Class Diagram



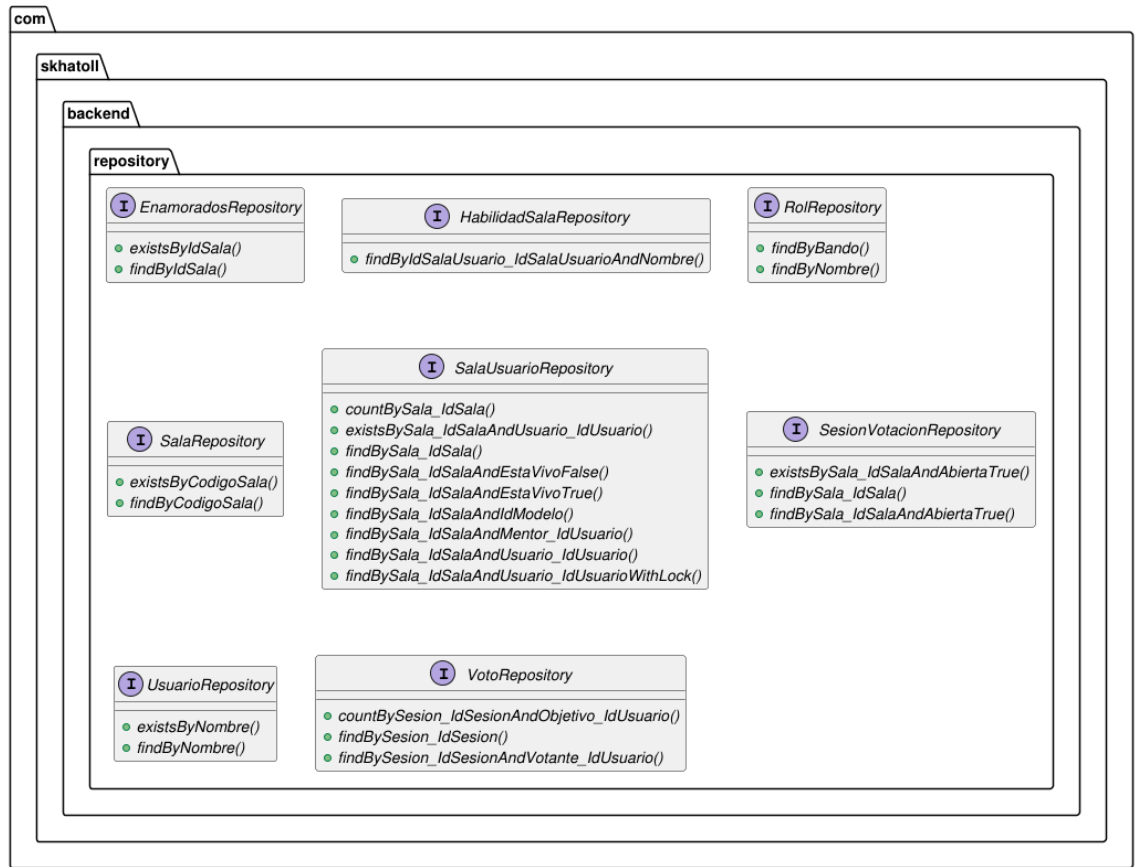
PlantUML diagram generated by SketchUML (<https://bitbucket.org/mesmeur/sketchuml>)
For more information about this tool, please contact philippe.mesmeur@gmail.com

PARTIDA's Class Diagram

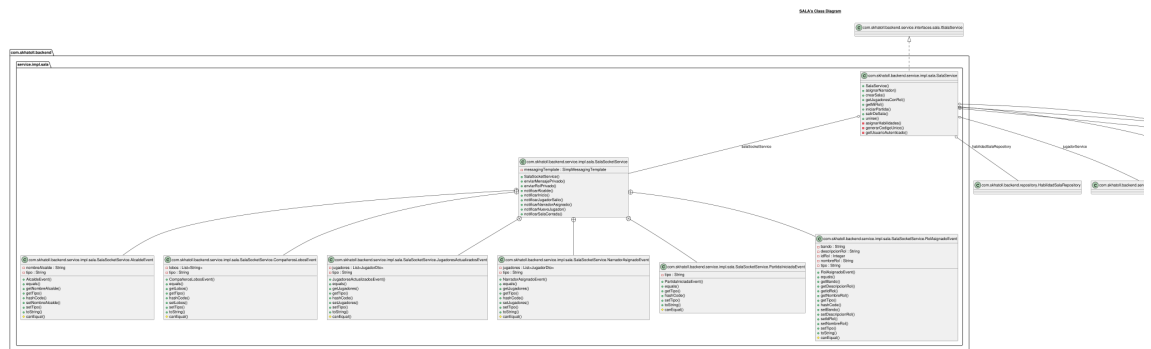


PlantUML diagram generated by SketchIt! (<https://bitbucket.org/pmesmeur/sketch.it>)
For more information about this tool, please contact philippe.mesmeur@gmail.com

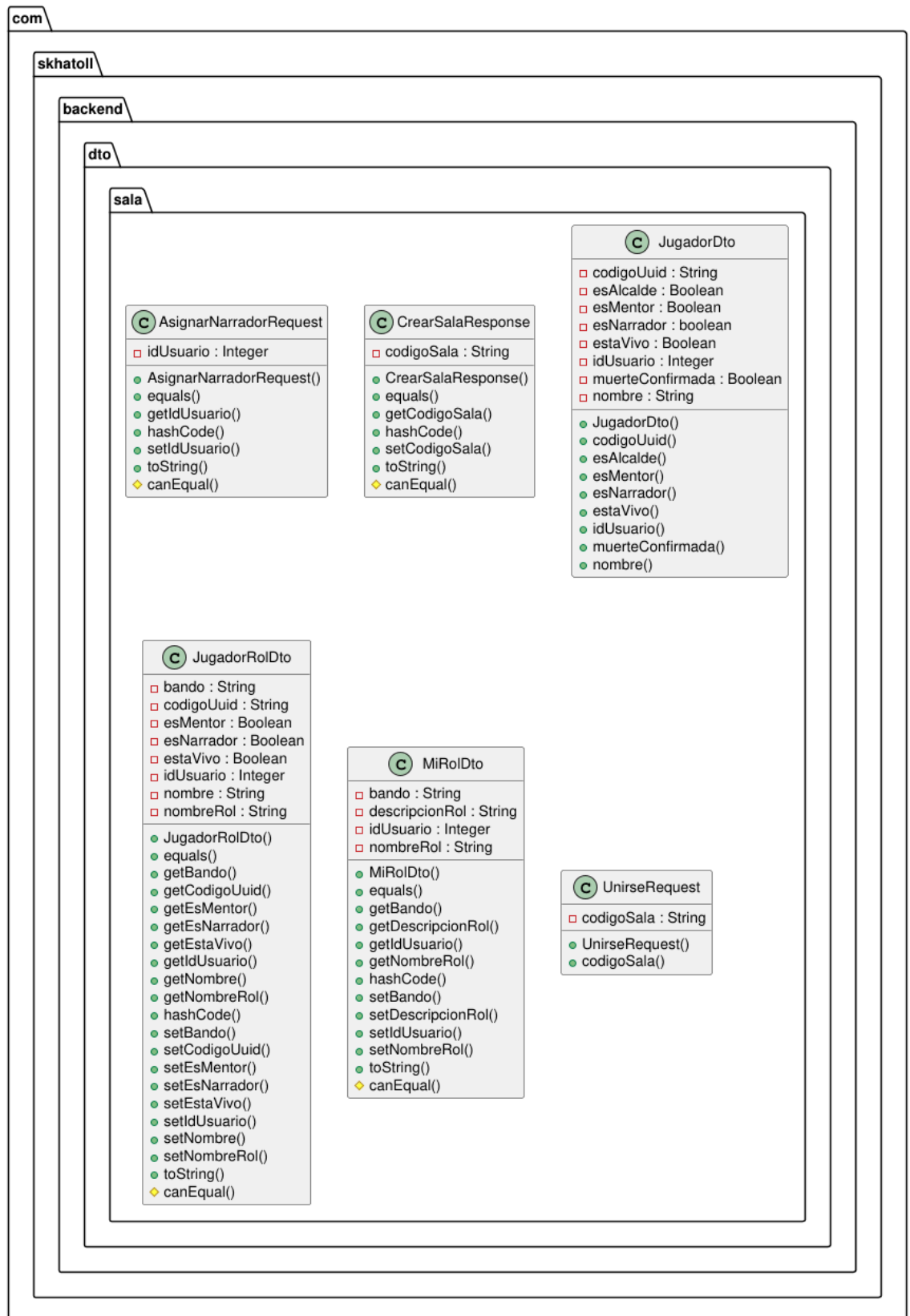
REPOSITORY's Class Diagram



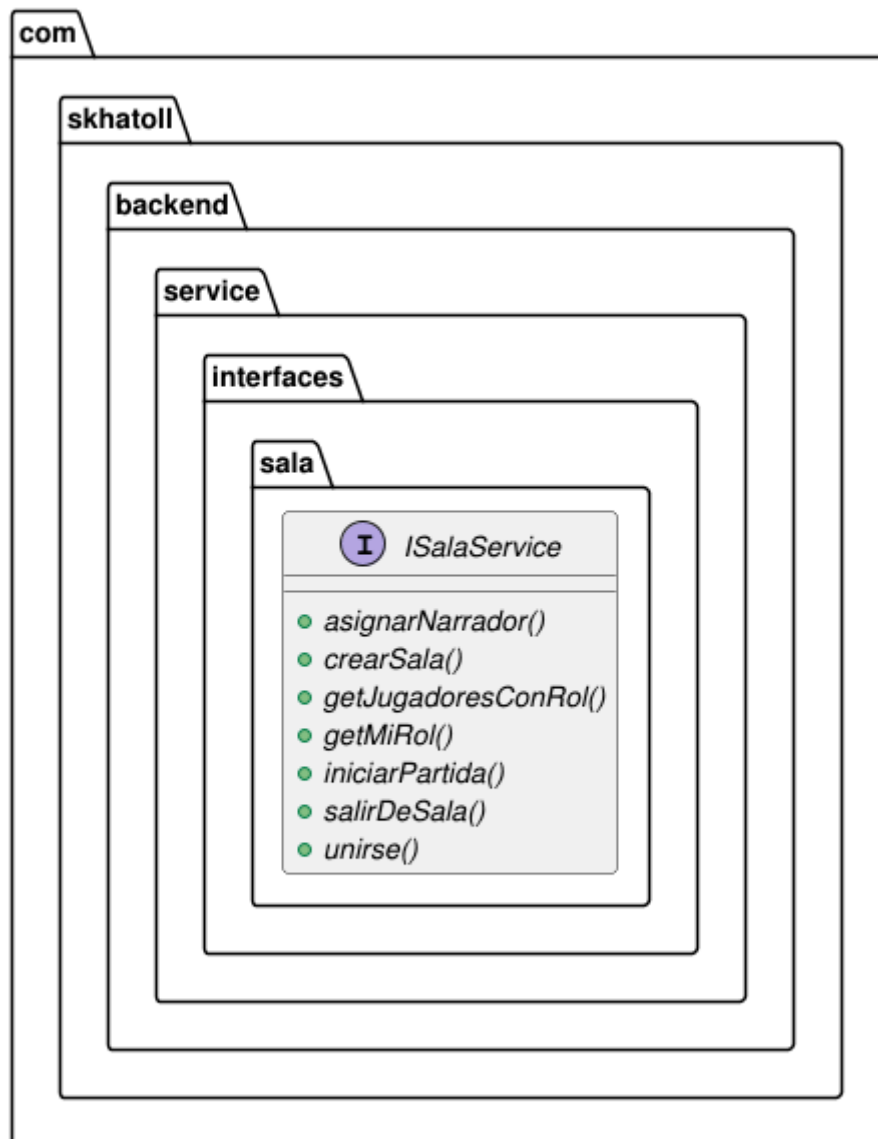
PlantUML diagram generated by SketchIt! (<https://bitbucket.org/pmismeur/sketchit>)
For more information about this tool, please contact philippe.mesmeur@gmail.com



SALA's Class Diagram

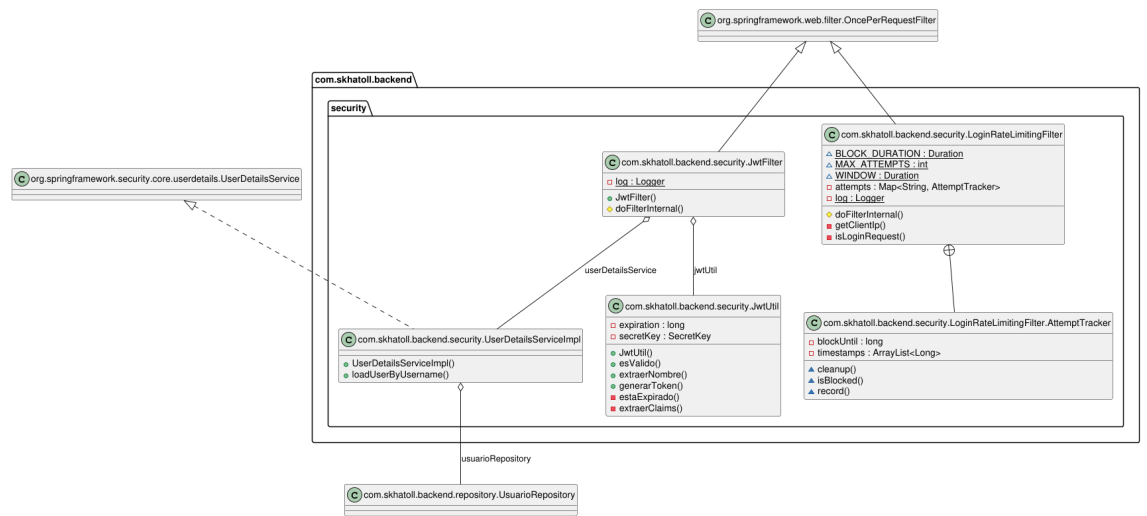


SALA's Class Diagram



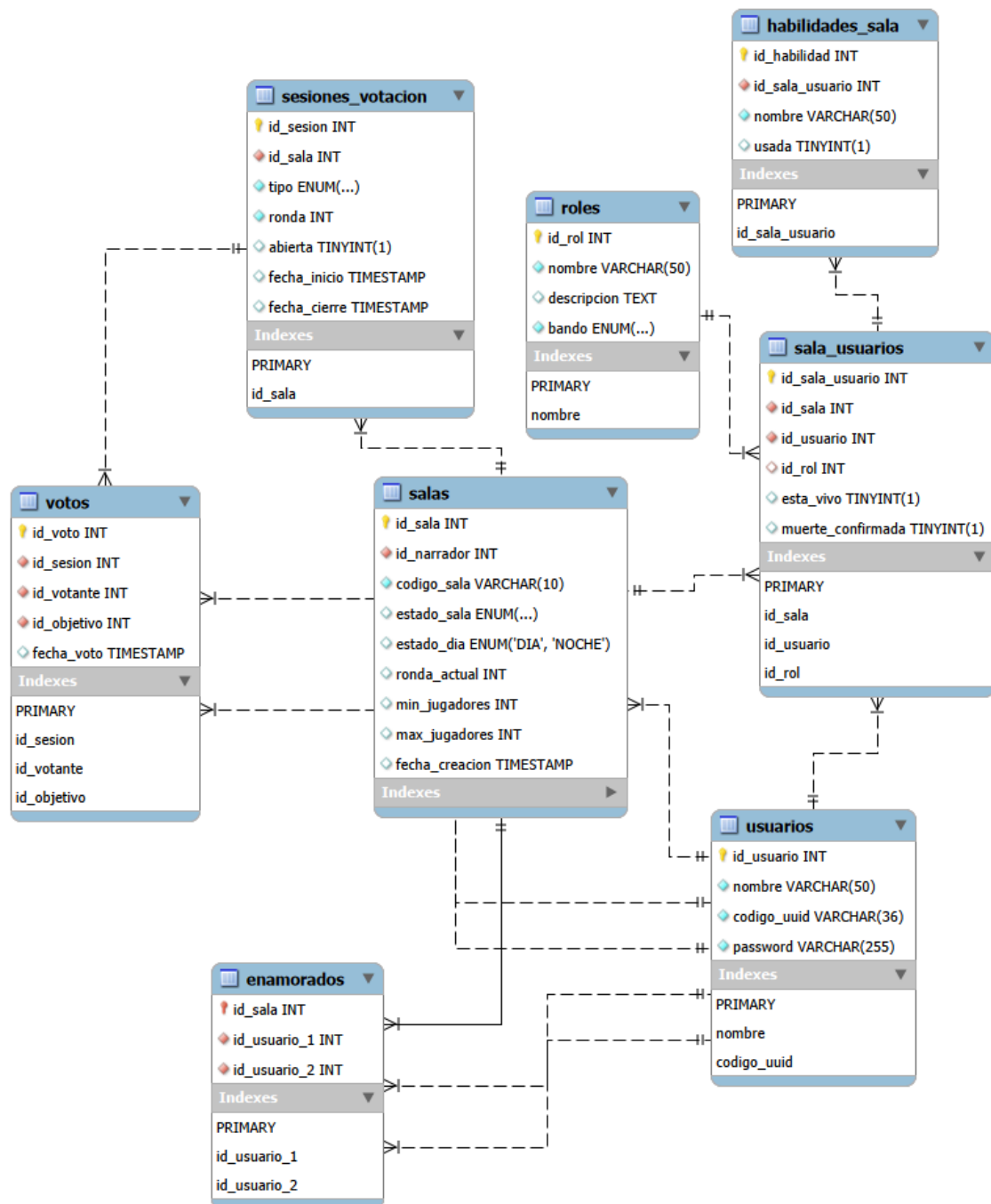
PlantUML diagram generated by SketchIt! (<https://bitbucket.org/pmesmeur/sketch.it>)
For more information about this tool, please contact philippe.mesmeur@gmail.com

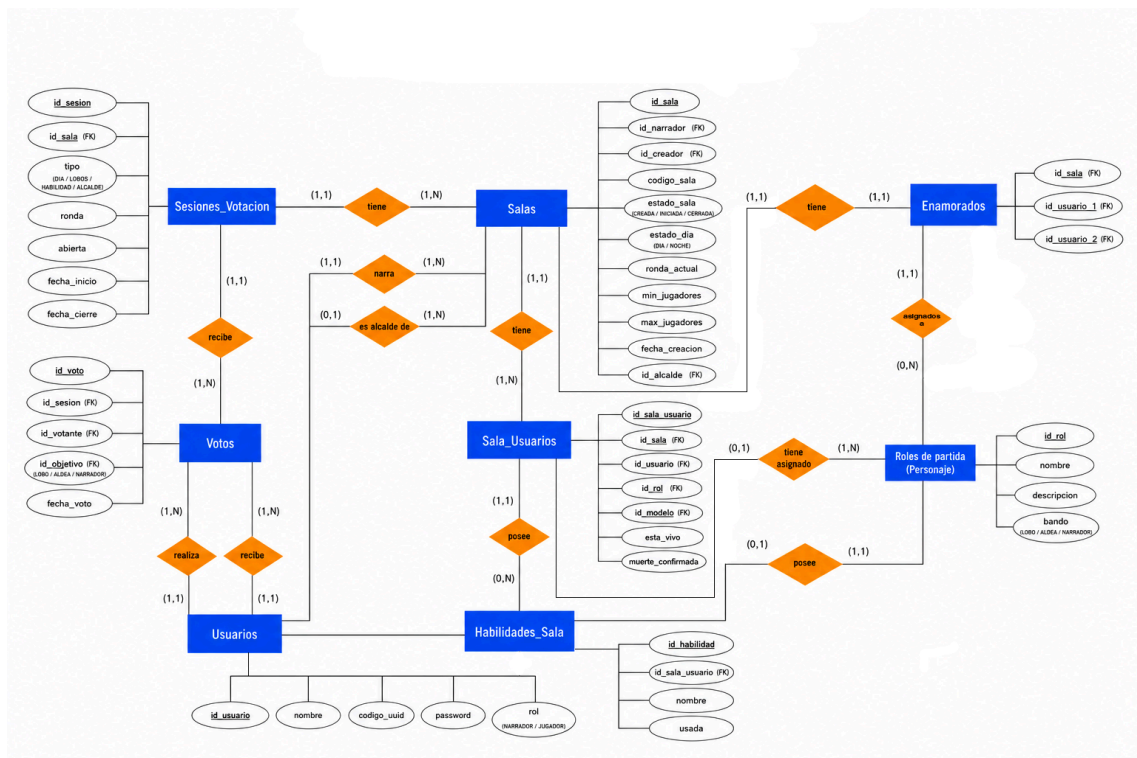
SECURITY's Class Diagram



PlantUML diagram generated by Bluewin (https://bluewin.ch/en/secure/secure/)
For more information about this tool, please contact philippe.meyers@bluewin.ch

5.3. Modelo de base de datos y Modelo entidad relación





6. Arquitectura de la aplicación

La arquitectura utilizada es la MVC (Modelo, Vista y Controlador) que está compuesta por 3 capas donde el controlador se encarga de hacer de intermediario entre la vista (interfaz que muestra la información al usuario) y el modelo (gestor de los datos y reglas de la aplicación) a la vez que gestiona las interacciones del usuario decidiendo qué datos necesita consultar y cómo deben mostrarse.

En nuestro caso el proyecto sigue una **arquitectura MVC desacoplada**, separando claramente el frontend y el backend, facilitando la escalabilidad, el mantenimiento y el trabajo en equipo.

Esta arquitectura permite tener una estructura del código más organizada y localizable además de facilitar la incorporación de nuevas funcionalidades sin afectar al resto del sistema, lo que permite una mayor escalabilidad, reutilización de código y mejoras a la hora de coordinarse para trabajar en equipo.

A todo esto se le añade que el MVC se adapta fácilmente a la mayoría de frameworks y tecnologías más utilizadas además de que se pueden hacer pruebas unitarias de manera más sencilla y modular.

Siguiendo este modelo, este proyecto está estructurado en distintos paquetes que están organizados según las funcionalidades de sus clases. Esto se verá mejor en el punto 6.1.

6.1. Estructura del proyecto

```
1  frontend
2  |
3  |─ index.html
4  |─ public
5  |─ src
6  |   │─ App.vue
7  |   │─ assets
8  |   │   │─ imgs
9  |   │   └─ styles
10 |
11 |   └─ components
12 |       │─ juego
13 |       │   │─ BotonMiRol.vue
14 |       │   │─ CabeceraJugador.vue
15 |       │   │─ IndicadorDiaNoche.test.js
16 |       │   │─ IndicadorDiaNoche.vue
17 |       │   │─ ListaPersonajes.vue
18 |       │   │─ ListaReglas.vue
19 |       │   │─ MesaJugadores.vue
20 |       │   │─ PanelControlNarrador.vue
21 |       │   │─ PanelVotacionesJugador.vue
22 |       │   │─ poderes
23 |       │   │─ roles
24 |       │   └─ ZonaPoderes.vue
25 |       │─ layout
26 |       └─ lobby
27 |
28 |
29 |   └─ composables
30 |   └─ data
31 |   └─ main.js
32 |   └─ plugins
33 |   └─ router
34 |   └─ services
35 |   └─ store
36 |       │─ GameEvents.js
37 |       │─ index.js
38 |       └─ modules
39 |
40 |   └─ test
41 |       └─ setup.js
42 |   └─ views
43 |       │─ CargaRolView.vue
44 |       │─ EliminadoView.vue
45 |       │─ EsperaNarradorView.vue
46 |       │─ InicioView.vue
47 |       │─ JugadorView.vue
48 |       │─ LobbyView.vue
49 |       │─ NarradorView.vue
50 |       │─ PersonajesView.vue
51 |       │─ ReglasView.vue
52 |       │─ ResultadosView.vue
53 |       └─ SalaView.vue
54 |─ vite.config.js
55 |─ vitest.config.js
```

```
1  backend
2  |─ pom.xml
3  |─ src
4  |   │─ main
5  |   │   │─ java (com.skhatoll.backend)
6  |   │   │   │─ BackendApplication.java
7  |   │   │   │─ config
8  |   │   │   │   │─ CorsConfig.java
9  |   │   │   │   │─ SecurityConfig.java
10 |   │   │   │   └─ WebSocketConfig.java
11 |   │   │   │─ controller
12 |   │   │   │   │─ AuthController.java
13 |   │   │   │   │─ PartidaController.java
14 |   │   │   │   └─ SalaController.java
15 |   │   │   │─ dto (Contiene subcarpetas Auth, Partida y Sala)
16 |   │   │   │   │─ AuthResponse.java
17 |   │   │   │   │─ EstadoPartidaDto.java
18 |   │   │   │   └─ JugadorDto.java
19 |   │   │   │─ entities
20 |   │   │   │   │─ Rol.java
21 |   │   │   │   │─ Sala.java
22 |   │   │   │   └─ Usuario.java
23 |   │   │   │─ exception
24 |   │   │   │   │─ ErrorResponse.java
25 |   │   │   │   └─ GlobalExceptionHandler.java
26 |   │   │   │─ repository
27 |   │   │   │   │─ RolRepository.java
28 |   │   │   │   │─ SalaRepository.java
29 |   │   │   │   └─ UsuarioRepository.java
30 |   │   │   │─ security
31 |   │   │   │   │─ JwtFilter.java
32 |   │   │   │   └─ JwtUtil.java
33 |   │   │   │─ service
34 |   │   │   │   │─ interfaces (IAuthService, ISalaService, IPartidaService...)
35 |   │   │   │   └─ impl (AuthService, SalaService, PartidaService, WebSockets...)
36 |   │   │   │─ util
37 |   │   │   │   │─ constants
38 |   │   │   │   │   │─ ErrorMessages.java
39 |   │   │   │   │   └─ GameConstants.java
40 |   │   │   │─ resources
41 |   │   │   │   └─ application.properties
42 |   └─ test
43 |       │─ java (Integration & Unit Tests)
44 |       │   │─ PartidaControllerIntegrationTest.java
45 |       │   └─ HabilidadServiceTest.java
46 |       └─ resources
47 |
```

6.2. Librerías externas utilizadas

Backend

- **Spring Boot starters** (Web, Data JPA, Security, WebSockets, Validation)
- **Lombok 1.18.38** - Generación automática de código
- **JWT 0.12.6** - Tokens JWT para autenticación
- **MySQL Connector 8.x** - Driver MySQL
- **H2** - Base de datos en memoria para testing

Frontend

- **Vue 3.5.28** - Framework principal
- **Vue Router 5.0.2** - Navegación
- **Vuex 4.1.0** - Gestión de estado
- **Bootstrap 5.3.3** - Framework CSS
- **Sass 1.99.0** - Preprocesador CSS
- **Axios 1.6.0** - Cliente HTTP
- **@stomp/stompjs 7.3.0** - Cliente STOMP
- **sockjs-client 1.6.1** - WebSockets
- **Vite 7.3.1** - Build tool
- **Font Awesome** (externo) - Iconos
- **Google Fonts** - fuentes utilizadas en la página

Testing

- **Vitest 2.1.8** - Framework testing frontend
- **@vue/test-utils 2.4.6** - Testing componentes Vue
- **jsdom 25.0.1** - DOM virtual
- **JUnit 5 + Mockito** - Testing backend

7. Manual de despliegue

Imágenes de Docker Hub:

- dgs0/skhatoll-frontend
- dgs0/skhatoll-backend

1. Descargar el archivo *docker-compose.yml*

Primero se debe descargar el archivo de configuración de Docker Compose desde el repositorio del proyecto.

Ejecuta el siguiente comando en la terminal:

```
curl -O https://raw.githubusercontent.com/David-GutSal/Skhatoll/develop/docker-compose.yml
```

Este comando descargará el archivo *docker-compose.yml* en el directorio actual.

Se recomienda crear previamente una carpeta para el despliegue del proyecto y ejecutar el comando dentro de ella, por ejemplo:

- mkdir skhatoll
- cd skhatoll
- curl -O https://raw.githubusercontent.com/David-GutSal/Skhatoll/develop/docker-compose.yml

2. Crear el archivo *.env*

Una vez descargado el *docker-compose.yml*, es necesario crear un archivo llamado *.env* en el mismo directorio.

Docker Compose utilizará este archivo automáticamente gracias a la instrucción:

```
env_file: .env
```

El archivo *.env* contendrá las variables de entorno necesarias para el funcionamiento del backend y la conexión con la base de datos.

```
ALLOWED_ORIGIN=https://deviancy-booted-urgent.ngrok-free.dev
```

```
DB_PASSWORD=AVNS_VfSu0SV217PwgX5hIVR
```

```
DB_URL=jdbc:mysql://ghw-server-proyectoatraco.f.aivencloud.com:13213/Skhatoll
```

```
DB_USERNAME=avnadmin
```

```
JWT_EXPIRATION=8640000
```

```
JWT_SECRET=zHDG707WYAZBpRatBXcWY747azdadnCO5BvwDMhMv8xrQSUyyQ1MkPE4y6md  
D
```

Explicación de las variables

- **ALLOWED_ORIGIN**
URL desde la que se accederá al frontend de la aplicación.
El backend permitirá solicitudes únicamente desde esta dirección mediante configuración CORS.

Durante la demostración del proyecto se utilizará un subdominio específico.
Este no se encuentra disponible para la revisión debido a que requiere autenticación mediante token.

- **DB_URL**
URL de conexión a la base de datos utilizada por la aplicación.
- **DB_USERNAME**
Usuario de acceso a la base de datos.
- **DB_PASSWORD**
Contraseña del usuario de la base de datos.
- **JWT_SECRET**
Clave secreta utilizada para firmar y validar los tokens JWT de autenticación.
Se recomienda utilizar una cadena larga y segura.
- **JWT_EXPIRATION**
Tiempo de expiración de los tokens JWT en milisegundos.

3. Iniciar los contenedores

Una vez configurado el archivo `.env`, se puede iniciar la aplicación ejecutando:

```
docker compose up -d
```

Explicación del comando

- *docker compose up*
Crea e inicia los contenedores definidos en el archivo *docker-compose.yml*.
- *-d*
Ejecuta los contenedores en segundo plano, permitiendo seguir utilizando la terminal.

4. Verificar el despliegue

- Tras iniciar los contenedores, la aplicación quedará disponible en:
 - `http://localhost`
- Para comprobar que los contenedores están funcionando correctamente se puede utilizar:
 - `docker compose ps`
 - Y para visualizar los logs:
 - `docker compose logs -f`

5. Detener la aplicación

- Para detener los contenedores sin eliminarlos:
 - `docker compose stop`
- Si se desea detener y eliminar los contenedores creados:
 - `docker compose down`